

01 - Interpretability

ML Algorithm Example: loan eligibility prediction.

i/p features: age, income, loan amount, credit history, loan usage

↳ challenges in this algo

- fairness & ethics in decision making.
- Preserving privacy. to protect personal data
- Robust algo [small changes in features shouldn't alter decision.
- Give explanations for decision to trust the algorithm.

can we trust & interpret how the decision is made?

Interpretability refers to degree to which we can understand the **cause** of a decision.

An interpretable ^{ML} model gives info on **why** certain decisions are made.

- important to know **why** the model outputs the way it does for high-risk applications.
- we define interpretability in context of **supervised learning**.

Categories of interpretability methods

based on model training

Pre-hoc / intrinsic

- design an ML model that is interpretable because of its simple structure
- Ex: Decision tree / linear reg. model.

Post-hoc

- interpretability can be performed to an already trained ML model.
- Ex: Deep Neural Net

Class of model

Model specific

- approach applicable to particular class of models.

Model-agnostic

- Approach is applicable to any ML model. These methods **normally** used with **already trained models**.
- **No access** to model internals, like n/w parameters.
- only access** to i/p & o/p.

Prediction

Local (sample based)

- explain individual predictions

Global

- explain entire model

Output of interpretability Methods.

- developing interpretability algo assumes to define i/p & o/p. to interpretability method
- i/p: Model features, Model parameters, data samples
- o/p of method:
 - Feature summary statistics. Ex: t-SNE on features
 - Model param. analysis & visualisation. Ex: kernel of a NN
 - Data samples ~~off~~ from training set or newly created that provide explanation. Ex: adversarial samples.

Aspects of interpretability Methods

Global interpretability

1) Complete Model

- understand how model works at once.
- assumed to have: access to ML algo, trained model & training data
- This interpretability method is not feasible in practice due to modern models being complex.
Ex: deep NN can't be understood ^{as a whole} given the pretrained-model training algo & the data.

2) Model parts (modules):

- understanding parts/modules in a global basis is easier.
- Ex: Consider a decision tree where goal is to understand a particular splitting function.

Local interpretability (most common)

1) Single prediction

- examine behaviour of model for single sample. For given i/p examine o/p.
- for sample under investigation we can perturb a feature & observe a prediction.
- in this way observe behaviour of features based on prediction

2) Batch of predictions:

- idea of single prediction can be applied for a group of samples.
- we can also consider different batches of predictions to generalise in a global interpretability level.

→ **Perturbation-based** methods popular as they **can be model agnostic**.

Explanation Properties

1) Explanation Properties based on **perturbation approaches**
(mainly focused on classification tasks)

- **Expressive Power**: refers to explanation language
Ex: Language can be → propositional calculus. Ex: if, or, and
→ Non-classical logic. Ex: many valued logic like fuzzy logic.
→ decision tree
→ linear model.
- **Translucency**: extent to which expl. method has access to ML model **internals**.

• It can be either

- **decompositional**: decompose model modules
- **pedagogical**: treat model as black-box
- **electic**: combination of above both.

- **Portability**: refers to range of ~~methods~~ models in which interpretability can be used.
- **Algorithmic complexity**: refers to **computational complexity** of the explanation algorithm.

2) Explanation Properties based on **quality of explanation**.

- **Accuracy**: ability to generalise to unseen samples.
- **Fidelity**: how well o/p explanations reflect behaviour of model.
Local fidelity (i.e. per sample) can't necessarily generalise to global level.
- **Consistency**: refers to similarity b/w explanations of 2 diff. models for **same sample**.
- **stability**: refers to similarity of explanations for **similar samples**.
- **comprehensibility**: how well do people understand the explanation.
- **certainty**: reflects confidence of o/p explanation.
- **representativeness**: refers to n.o. of samples ^{the} explanation covers.

Good explanations

- we need an explanation that includes reason that led to the result.
- contrastive expl.: need an expl. based on a reference.
Ex: another person with accepted loan so we understand why my loan was rejected.
- 1 to 2 causes are sufficient for human-friendly expl.
- good expl. includes unexpected causes that led to result.

Evaluation of interpretability

- there is no clear protocol on how to measure interpretability of a method & how to evaluate interpretability methods.

3 types of evaluation

which method is resource consuming?
" " " complex to implement?

1) Application-grounded evaluation:

Provide interpretability o/p along with model prediction, to a human expert for evaluation.

2) Human-grounded eval:

- Experiment conducted w/o experts.
- Approach useful when it is complex to involve large n.o. of experts.
- Human is given pairs of explanations & has to choose one with the highest quality.

3) Function grounded eval:

- No human involved
- Proxy defined & then the interpretability o/p is evaluated WRT proxy.
- A human is required though prior to evaluation to assess validation of the experiment. $\rightarrow$ proxy-task
- Most ML models use this.

Ex: check improvement in prediction of a model that has already been shown to be interpretable by human expert.
 $\rightarrow$ in this eval it is not easy to define proxy.

Linear Regression

- approach that computes weighted sum of i/p features.

i/p feature vector: $x \in \mathbb{R}^D$; $D \rightarrow$ dimension of x

o/p scalar value: $y \in \mathbb{R}$
(target)

$$\hat{y} = f(x; w) = w^T x + b$$

$\downarrow$ Predicted value $\downarrow$ $w \in \mathbb{R}^n$ (weights) $\downarrow$ bias/intercept

; goal: $\hat{y} \approx y$

- parameters w, b learned using training set, with m samples.

$$X^{\text{train}} \in \mathbb{R}^{m \times n}, \quad y^{\text{train}} \in \mathbb{R}^m$$

- test set can have diff. dim. than training set.

- learning uses loss func.

$$\mathcal{L}(f(x; w), y)$$

$\rightarrow$ performance measure of how good prediction is when learning param. on training set.

Ex: MSE

goal: min. MSE on train-set such that it results in low MSE on test set.

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i^{\text{train}} - y_i^{\text{train}})^2 = \frac{1}{m} \sum_{i=1}^m (w^T x_i^{\text{train}} - y_i^{\text{train}})^2$$

$\hookrightarrow$ assume b is included in w for simplicity.

- minimising loss $\equiv$ finding best w^* such that
 $w^* = \underset{w}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m (w^T x_i^{\text{train}} - y_i^{\text{train}})^2$

- 2 methods to solve for w^*

1) Normal eqn: - closed form sol.

- set grad. of loss to zero & solve for w^*
- 1 step algo (analytic approach)

2) Iterative:

- gradient based optimisation
- iteratively find param. that minimise MSE.

- MSE using matrix notation

$$\text{MSE} = (X^{\text{train}} w - y^{\text{train}})^T (X^{\text{train}} w - y^{\text{train}})$$

$\hookrightarrow$ skip $\frac{1}{m}$ as it has no impact on optimal param. values

- Expand & set grad. to zero, we get

$$w^* = (X^{(\text{train})T} X^{(\text{train})})^{-1} X^{(\text{train})T} y^{\text{train}}$$

Interpretation

How to interpret weights of linear regression?

- Linear relationship b/w i/p & weights makes model easy to interpret.
- Features with high weights have more influence to o/p & vice-vers.
- Feature importance: measured by absolute value of **t-statistic**.
t-statistic is ratio b/w estimated weight & its standard error. $\rightarrow$ how is standard error calculated?
larger it is = more int it is.
- visualisation of linear regression model is easy & intuitive.
- weight plot can be used to examine their importance. Ex: box plot.
- individual predictions can also be plotted Ex: features & outcome.

Lasso Regression

- used for eliminating less important features

- Linear reg. with L1-regularisation.

$$\text{loss func: } \frac{1}{n} \sum_{i=1}^n (w^T x_i^{\text{train}} - y_i^{\text{train}})^2 + \lambda \sum_{j=1}^p |w_j|$$

$\downarrow$
can be skipped

$\lambda \rightarrow$ controls influence of regulariser

- L1 reg. is good at bringing some of i/p features to 0.
[Feature elimination]
- Model can hence have sparse parameters.

Adv. Linear Reg.

- weighted sum o/p is easily interpretable
- visualisation of parameters is easy, as well as computing statistics.

Disadv. Linear Reg.

- cannot model complex & non-linear relationships b/w features & o/p.

Local Model-Agnostic Approaches

These approaches rely on individual prediction to provide explanation

Few std. approaches

1) Individual Conditional Expectation (ICE)

- plots that show how changing a feature affects the o/p.

Ex: predicted loan prob. v/s person age.

- 2) Local interpretable model-agnostic explanations (LIME)
- Explain individual predictions of a black box model.
 - A surrogate model is trained to approximate predictions of the black box model.
- 3) Counterfactual Explanations
- look for features to be changed to provide wished o/p.
 - challenge is to have an approach that performs better than trial & error.
- 4) Shapely value
- Assume that each feature is a player in a game where the model prediction corresponds to the payout, shapely values show how to fairly distribute the payout among players/features.

Local interpretable model-agnostic explanations [LIME]

- idea: approximate the black box ML model locally (per sample) using a surrogate model, which is a simple & interpretable model.
- approach based on perturbing i/p features of a data sample.
Ex: hiding image pixels. Perturbing original sample several times, a new dataset is created with perturbed samples.
 - The new predictions are computed for perturbed training data, while also being weighed based on their similarity to original data.
 - Using perturbations, predictions & computed weights, the surrogate model is approximated by a simple interpretable model, ex: linear regression or a shallow decision tree.

Algorithm:

- 1) Pick up a sample & black box ML model
- 2) Perturb the sample multiple times & then get prediction for each perturbation from black box model.
- 3) Weight each perturbed sample based on each distance from the original sample.
- 4) Train interpretable model on perturbations, including weights & predictions.
- 5) Rely on interpretable model to provide explanations.

Ex: image classification task. using CNN

- Top 5 predictions from model are obtained.
- seek explanation for top 1 op.
- rely on superpixels for perturbation instead of removing individual pixels.
 segments of superpixels.
- randomly choose to remove n.o. of super-pixels. [Perturbation]
 each perturbation is a new training sample.
- perform several perturbations Ex 300.
- obtain prediction for each perturbed sample using CNN model.
- compute dist. b/w orig. image & every perturbed image.
 Ex: cosine similarity.
 ↳ convert the dist. to a weight using some kernel function.
- fit weighted linear regression model to the perturbations, top 1-predictions & weights of perturbed data samples.
- model has same n.o. of parameters as superpixels & hence expect same n.o. of coefficient values.
- value of each coeff. tells us the importance of the superpixel. [linear reg. is interpretable]
- select top 4 coeff. ex: with highest value → these ^{correct} 4 superpixels lead to top prediction of ex: photocopier.

What is limitation of using Linear interpretable model

Properties

- perturbations for text could be removing words.
- algo is expensive due to perturbations.
- large n.o. of perturbed samples help achieve better model fit.
- LIME isn't stable. Produces diff. results for very similar samples.
- can be applied to any kind of ML model, not only NN.

O2- Attention & Transformers

Language Modeling

- Sequence modeling is de-facto approach in NLP.

- RNNs are common approach to model sentences text.

↳ face limitations due to challenges in modelling sequences, i.e. in seq. modeling we deal with long & short dependencies depending on sentence length.

* - in addition i/p & o/p sequences have dynamic length.

↳ ^{RNNs} used to be the std. approach for language modelling.

- LSTMs & GRUs & other approaches addressed the challenges of long sentences, but failed due to n.o. of words in a long sentence.

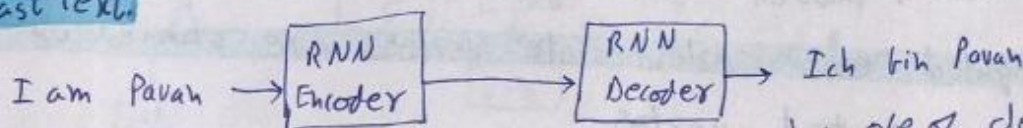
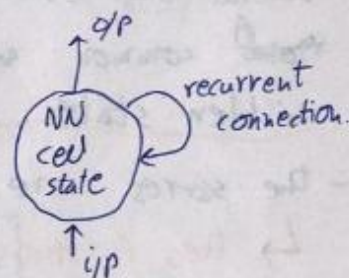
- Inspired by humans, a focus mechanism was necessary for focussing on relevant words.

↳ Motivation for introducing Attention to RNNs, for language processing.

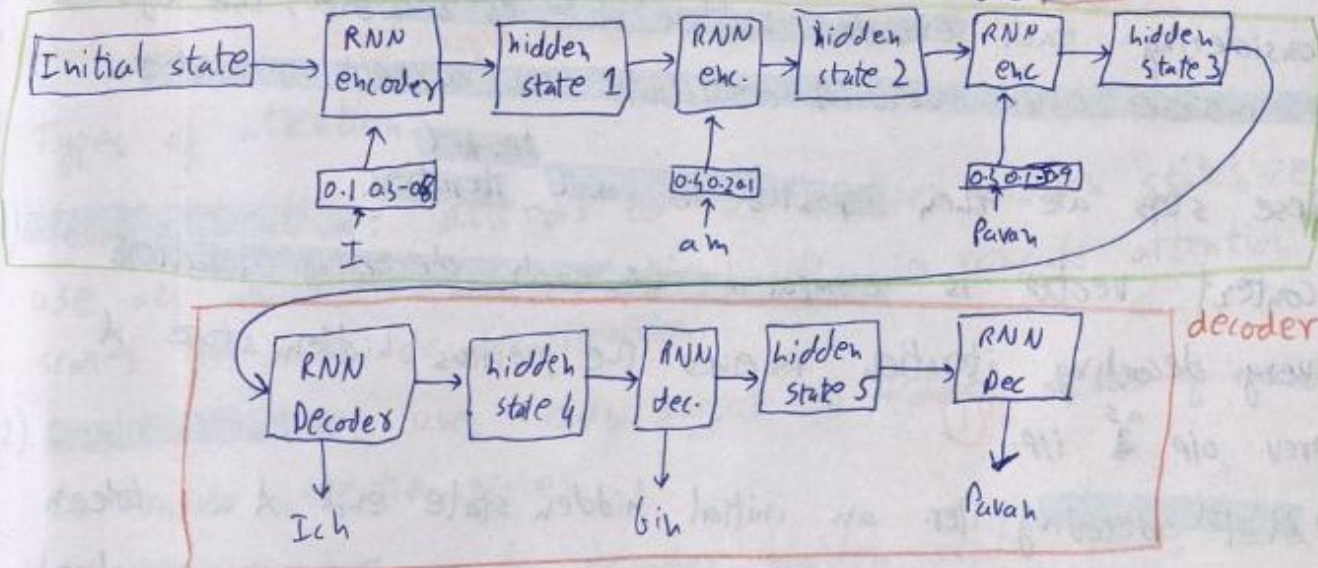
- seq. modelling for language needs a encoder to model i/p (sentence) & decoder to model the o/p (translated sentence).

↳ i/p is first transformed to an embedding.

↳ each i/p word is transformed to a vector using approaches like GloVe / fastText.



↳ o/p of decoder summarises the context



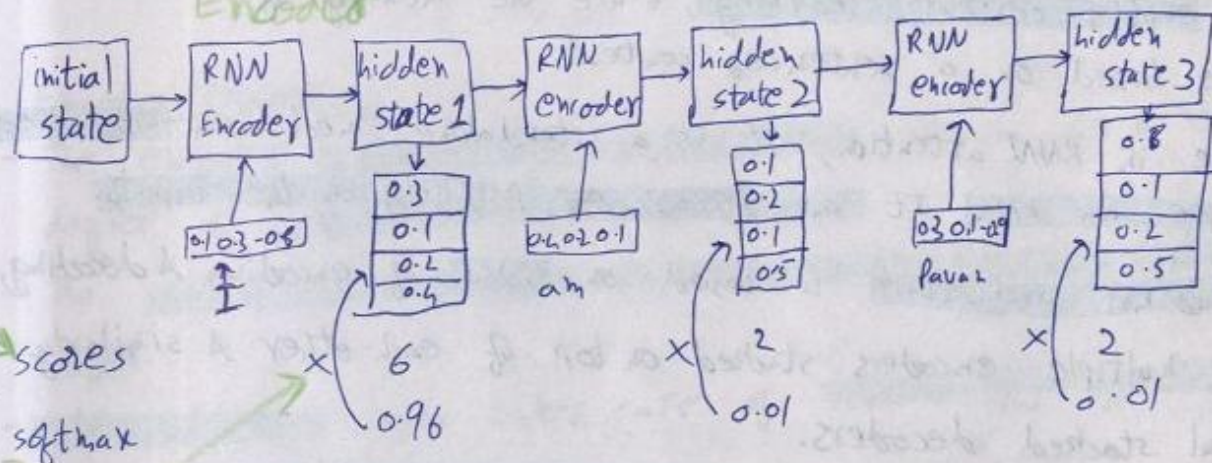
- Challenging to model long sentences based on only context info

↳ sol: use intermediate connection from enc. to dec. like enc.-dec. models for vision

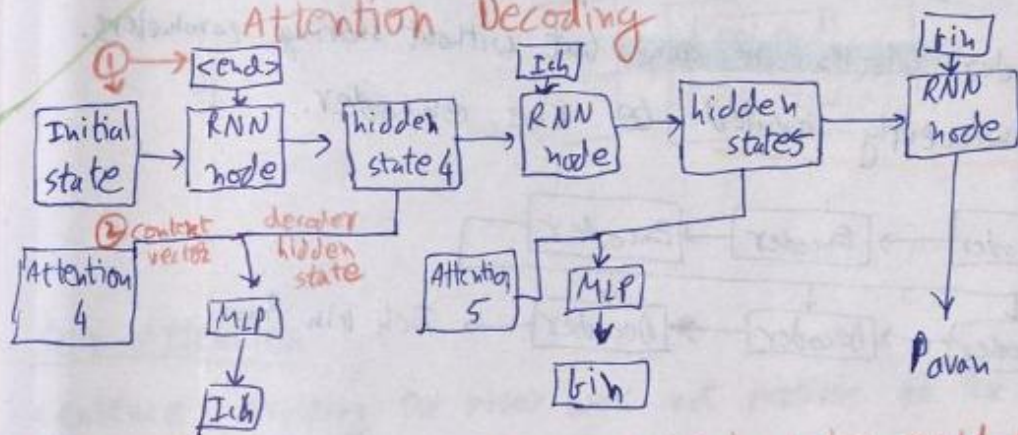
- Motivation behind attention mechanism is to provide more info. from enc. to dec.
- Consider passing all hidden states instead of just the last one & using them at each decoding step, to provide additional information to the decoder.
- Attention decoder (A-RNN) incorporates weighing mechanism to decide how much info to withhold from the hidden states.
- This weighing mechanism indicates how much to attend to the hidden state.
- At each decoding step the A-RNN looks where to focus given the encoder's hidden states.
- For each encoder hidden state, a score is computed. Several ways exist to define a score function.
~~most~~ A common way is to multiply the dot product of the hidden states of the encoder & decoder. (decoder hidden state at current iteration)
- the scores are then converted to probabilities using softmax.
↳ these functions make attention trainable as softmax is differentiable.
- each encoder hidden state is weighted by each softmax prob. hence each encoder hidden state get a different importance.
- finally weighted enc. hidden state vectors are summed up to form the new context vector.
- considering each encoder hidden state as a word, the softmax o/p indicate which word(s) contribute to current prediction.
- These steps are then repeated for each ^{decoder} iteration.
- context vector is computed for each decoding iteration.
- every decoding iteration receives the previous hidden state & prev. o/p ^{as} i/p. ①
- ↳ At 1st decoding iter. an initial hidden state end & a token are used to initialise the process. For lang. <END> is placed at the beginning.

- context vector computed from attention at current stage & computed decoder hidden state are concatenated & passed through a feed-fwd network^(MLP) to predict o/p. ②
- similar to standard RNNs, attention decoder has weights for hidden state vector & i/p vector.
 $\rightarrow$ prev. i/p token at current decoding step.
- Attention decoder learns to align i/p with o/p sequences.

Encoder



Attention Decoding



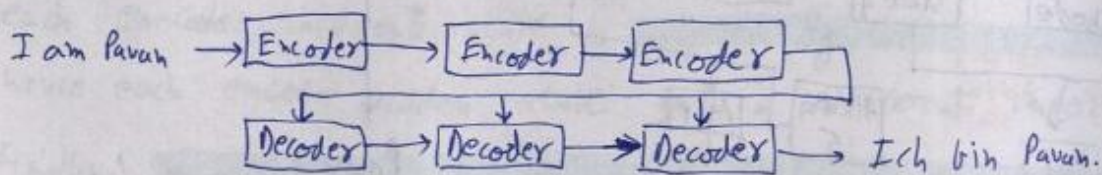
$\hookrightarrow$ some randomness exists as initial state could be anything.
What are the steps repeated at each decoder iteration?

Types of attention

- 1) **Global attention**: attends to all encoder hidden states, i.e. use all available encoder hidden states to compute attention scores for each decoder iteration.
- 2) **Local attention**: uses only subset of hidden states of encoder to compute attention scores.
- 3) **Hard attention**: each decoder hidden state relies on a single encoder hidden state.

The transformer

- Sequential processing is the **bottleneck** of the RNN approach.
- Transformer architecture allows **parallel processing** of the **encoding part** while also presenting idea of **self attention**.
- **self attention**: is a mechanism for **relating** the input embedding to remaining inputs of the sentences. It **enables context learning**, since the meaning of a word varies based on a sentence context.
- Unlike in RNN attention, it is a standalone mechanism **w/o need to use an RNN**. It can **process** ~~or~~ **parallelly** on the inputs.
- **Transformer architecture** is based on **idea** of encoding & decoding with multiple encoders stacked on top of each other & similarly several stacked decoders.
- All of the layers **do not share weights**.
- each encoder/ decoder **blocks are same** but without sharing parameters.
- every word is **independently** inputted to the encoder.

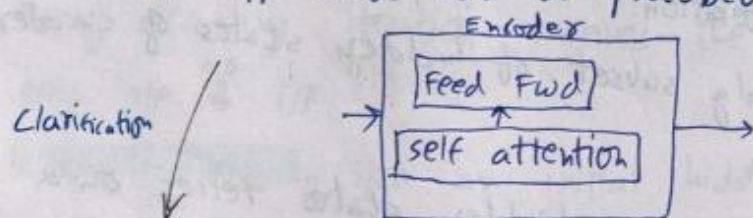


Encoder block

- consists of a **self attention layer** followed by **feed-fwd n/w. layer**.
- for the processed word, **self attention layer** helps to **attend the rest of the words of the sentence**.

Note: each word is processed **independently** by feed-fwd n/w.

All i/p words can be processed **together** by the encoder.



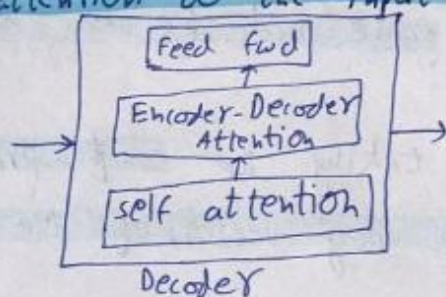
independently = no cross-word interaction in FF

together = 11th computation

FFN applies same weights to each token vector, & each vector is processed independently. No interaction b/w words (unlike in RNN), hence encoder can process all words/tokens together at once in 11th as a single matrix operation (unlike in RNNs).

Decoder Block

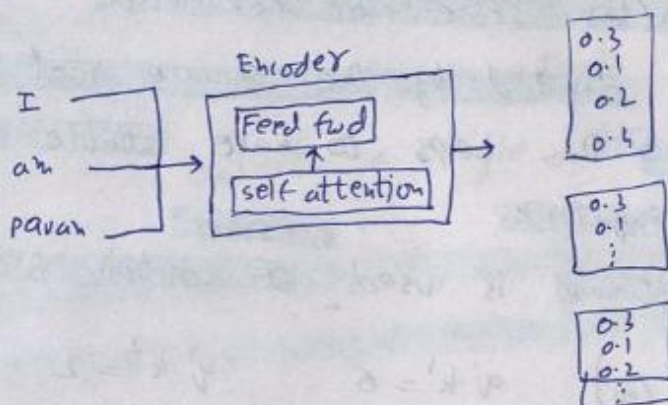
- consists of a **self attention layer** & **feed-fwd n/w layer**.
- in addition it has a **encoder-decoder attention layer** which implements the **attention to the input words**.



Encoder Processing

- The encoder **processes all words at once**, allowing for a high degree of **parallelisation in the encoder**.
- The **o/p of each encoder layer move to the next encoder layer**.
- **self attention layer** takes care of **relations b/w i/p words**.

Ex: I am Pavan.



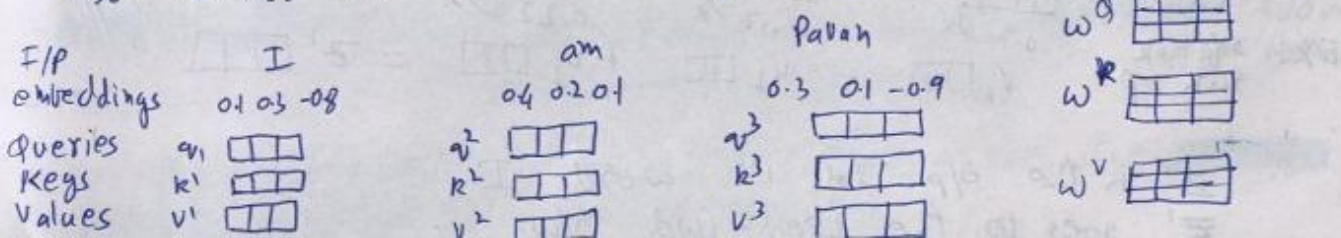
Self-Attention

sentence: crossing the river was not possible for the turtle due to its exhaustion.

its refers to turtle/river?

- self attention is the **mechanism to focus on the relevant words of the sentence**.
- For **each i/p word 3 vectors are created: Query, Key & Value**
- each of the vectors is obtained by **multiplying input embeddings with 3 matrices**.

These matrices are learned during model training.



scores

- a score is computed similar to attention mechanism.
- for each word, we need to calculate a score compared to every other word in the sentence.
- for the current word, score indicates how much attention to put on other words.
- score is computed by taking the dot product of the query vector with the key vector, of the corresponding word.

- For ex on prev. page

- score (q^1): $q^1 \cdot k^1 = 30$ $q^1 \cdot k^2 = 60$ $q^1 \cdot k^3 = 10$
- score (q^2): $q^2 \cdot k^1$ $q^2 \cdot k^2$ $q^2 \cdot k^3$
- ...
so on.

Normalisation

- follows after scores are calculated
- scores are divided by the square root of the dim. of the key vector. This helps to have stable gradients during back propagation.
- finally softmax is used ^{over scores} to obtain probabilities.

Ex: score (q^1)	$q^1 \cdot k^1 = 6$	$q^1 \cdot k^2 = 2$	$q^1 \cdot k^3 = 4$
Normalise	$\frac{6}{\sqrt{3}}$	$\frac{2}{\sqrt{3}}$	$\frac{4}{\sqrt{3}}$
softmax	0.7	0.07	0.22

How to interpret softmax scores?

- After softmax is calculated value vectors are weighted with the softmax scores.
- The weighted value vectors are then summed up to form output of the self-attention layer.
- This process is repeated for each word.

Ex: $\begin{matrix} \text{Values} \\ \text{softmax} \\ \text{sum up} \end{matrix}$

$\begin{bmatrix} & & \end{bmatrix} \times 0.7$	$\begin{bmatrix} & & \end{bmatrix} \times 0.07$	$\begin{bmatrix} & & \end{bmatrix} \times 0.22$	$= z^1 \begin{bmatrix} & & \end{bmatrix}$
$v_1 \begin{bmatrix} & & \end{bmatrix}$	$+ v_2 \begin{bmatrix} & & \end{bmatrix}$	$+ v_3 \begin{bmatrix} & & \end{bmatrix}$	

z^1 is the o/p for 1st word "I".
 z^1 goes to the feed-fwd n/w.

- The process examined is for a single token (word). But this is a slow process & to accelerate it, all calc. are done in matrix form.

X (input embedding matrix) $\times w^q = Q$ $\times w^k = K$ $\times w^v = V$

self attention (matrix form)

$$\text{softmax} \left(\frac{Q \times K}{\sqrt{d}} \right) \times V = Z$$

$d \rightarrow$ dimensions of the key vector.

Multi-head attention

- 1 set of Q, K, V matrices form one head of attention
- using multiple heads helps us focus on different positions.
- we will then have $z_1, z_2 \dots z_N$ attention outputs.
- But, the feed fwd n/w expects a single attention o/p of fixed size.
- Hence all attention heads $z_1, z_2 \dots z_N$ are concatenated & multiplied with an additional weight matrix to condense to a single Z .

Positional encoding

- since there is no sequential encoding of i/p, word order is missing from encoder.
- to solve, positional vector is added to word embeddings before transformer network processing begins.
- The positional encoding will be used both from the encoder & decoder.

Residual connections & layer Normalisation

- encoder additionally has residual connections & layer norm (Add & Norm)
- these operations are a part of the decoder too.
- the encoder is repeated several times & then the o/p of last encoder is used by the decoder.

Transformer Decoder

- last encoder produces set of key & value matrices used by decoder in encoder-decoder ~~block~~ attention layer.
This is another multi-head self attention layer & the query matrix comes from the previous decoder layer.
- enc-dec. attention layer helps to attend the important i/p words for producing an o/p at current position.
- o/p of decoder is used as i/p to next decoding iteration.
Iter. stop once specified symbol has been predicted, Ex: <END>.
- The i/p to decoder is transformed to an embedding & pos. encoding is added to it.
Then it is passed through ~~masked~~ ^{self-}multihead attention block.
This is a masked attention block, i.e. attend to only previous positions & not to the following ones.
- o/p of last decoder layer is a vector.
^{To output} ~~For~~ ^{prob} of each word, a linear layer (MLP) & a softmax layer follow.
- softmax function corresponds to o/p vocabulary size. Ex: 30k words & symbols.

Note: Transformer decoder processes its inputs sequentially.

(BPE)

- sentence encoding: Byte-pair encoding tokenization.
- Cross-entropy as loss function can be used for training.
- Transformers are SOTA model for language models.
- Transformer architectures for diff. data modalities cropped up.
- Transformer models are large & complex models.
Training requires special infra.
- Model inference isn't realtime w/o model compression.

Vision transformer

how is it diff. from language transformer?

- ViT model originally in paper was ~~only~~ an encoder only architecture as it primarily was for image classification, where goal was to map an image to a single o/p label.
- Language transformer takes a seq. of tokens/words as i/p & converts it into a numerical vector/embedding. These embeddings are then fed into transformer's encoder blocks.
- in contrast ViT
 - first divides i/p image into patches (non-overlapping grid) of squares
 - flattening & proj: each patch is flattened into a 1D vector & then projected into a higher dim. space to create patch embedding. (similar/analogous to word embeddings)
 - positional & class embedding: to maintain spatial info, positional embedding is added to each patch embedding indicating its orig. location in image.
Additionally a learnable embedding [class] is prepended to seq. of patch embedding. This class/token's final o/p will represent image's overall classification.
 - once done, the seq. of embedded patches are fed into transformer encoder.
 - encoder o/p is then passed to MLP head & for final classification task.

03- Self-supervised Learning

- supervised learning works well in practice when there are enough annotated samples, but sample annotation is a costly process & is not possible for every ML task.
- Furthermore, samples available for specific applications may be limited causing poor generalisation.
- Is there a way to achieve good supervised learning perf. when available data is insufficient to train?
- Transfer learning: is the approach of adapting the pre-trained model to the new task using the new dataset.
Ex: Pre-train a deep neural network to imageNet dataset for classification & then use this model for transfer learning.
- i.e. we use pre-trained model weights as initialisation for the new (downstream task)
 - Pre-training is normally done with large-scale dataset.
 - The pre-trained model (backbone) is modified with a new task for the downstream task.
- The deep-NN is entirely / partially trained on dataset & task of interest using the optimised weights from pre-training as prior knowledge.
- Performance drops if the new from downstream task is used again on the main task.
- Observations:
 - process of training the pre-trained model on the task in place (downstream task) is called fine-tuning.
 - fine-tuning often requires different set of hyper-parameters. Ex: Learning rate. Optimiser remains same, may have to change momentum & start from 0.
 - can reduce training time as we start the optimisation from a better set of initial parameter values.

Pre-training Limitations

- effective transfer learning comes at a cost of pre-training with large-scale labelled datasets.
- pre-training on large-scale datasets like ImageNet might not generalise well to other domains such as medical image analysis.
 - ↳ domain & task misalignment can cause pretrained model not to be well suited.
- hence transfer learning might not always lead to good performance for downstream task.
- Pre-training cannot prevent the task of annotating the dataset of interest. It is a costly process.

Can we replace supervised pre-training with another task to obtain a good model initialisation?

sol: Training with unlabelled images

- huge unlabelled data available on the internet. Web scraping can easily create datasets that contain only training samples but no labels.

Objective: train deep-NN using unlabelled data to learn useful representations (features) that ^{would} benefit the target/downstream learning task.

Self-supervised learning is a paradigm where supervision stems from the data itself.

- idea: derive supervision from a proxy/surrogate task.
- Proxy tasks allow definition of a differentiable loss func. & hence provide necessary supervision for training the model.

↳ clustering is similar to classification w/o labels. such tasks can be used for self supervision.

How is SSL diff. from unsupervised learning?

↓
model creates its own supervising sig. from the structure of the data

(BERT, simCLR...)

↓
model is trained on unlabeled data & uses no supervision. it just looks for patterns/comparisons directly.

(K-means, AutoEncoders, PCA)

SSL-workflow

- 1) Pre-train backbone (encoder) on the pretext task with unlabeled data.
 - 2) Fine-tune backbone on downstream task by freezing the backbone weights & training only the head.
- optimise parameters of the backbone θ by training on the pretext task using differentiable loss func.

$$L_{\text{pretext}}$$

$$\theta^* = \underset{\theta}{\operatorname{argmin}} L_{\text{pretext}}(x; \theta)$$

- then we rely on same θ^* to initialize the same backbone model to learn downstream task, & either fine-tune task head only or the entire n/w.

Downstream task also has a differentiable loss func. $L_{\text{downstream}}$ with optimisation defined as

$$\zeta^* = \underset{\zeta}{\operatorname{argmin}} L_{\text{downstream}}(x; \theta^*, \phi)$$

$\phi \rightarrow$ task head parameters

$\zeta^* \rightarrow$ optimised parameters of the model.

PS: backbone = encoder

Pretext tasks:

- pretext tasks motivated from images discussed.
- proxy / pretext / surrogate task mean same
- pretext tasks provide us with labels for free
- we do not care about performance in pretext task, instead we focus on learning a useful representation for the target task.
- we do not aim to make loss on pretext task to zero as we don't want to overfit on it.

Pretext task: surrogate classes

- set of exemplary image patches sampled from existing dataset in a random fashion
- image gradients are used to locate informative patches that contain parts of an ~~image~~^{object} or texture in general.
- after sampling an informative patch, it undergoes through a composition of geometric & photometric transformations.
- each composition of transformations creates a new image patch that belongs to same surrogate class with the original image patch.

transformations: translation, rotation, scaling, contrast, color transformation.

- Based on number of patches N , there will be N diff. surrogate classes.

Task: Discriminating b/w set of N surrogate classes is the pretext class.

- Backbone w/o can be combined with FC layer & a o/p function to be trained. The loss function L_{pretext} can be -ve loglikelihood.
- Afterward backbone can be trained on the target task
Ex: image classification.
- Pretext performance is not always relevant to perf on target task.

5 Pretext task: Patch relationship

- 1) - Uses image context in forms of patch information to learn a visual representation.
- 2) task: sample an image patch & a neighbouring patch after defining an image grid.
Then predict their relation within the grid.
- both patches are processed by the same CNN to predict their relationship as a classification problem.
- The relationships can be discretised into certain no. of classes.

Algorithm

1. randomly sample a patch from the image
 2. set up a grid with this random patch placed in the center.
 - To avoid trivial sol. ex: same texture across 2 patches, add enough pixel gap b/w grid cells.
 - in addition add pixel augmentation (down/up-sample) as well as jitter in pixel location.
 - color correction is also helpful to force the n/w not to memorise the patches.
 3. Given the reference patch & a randomly sampled patch the n/w then predicts where the randomly sampled patch is among the 8 neighbour locations.
- By learning image context with pretext task, model should be able to learn a backbone that we can use for a concrete task.

Pretext task: Jigsaw puzzle

- multiple patches are extracted in a grid format from the same image.
- The patches are then shuffled & the model learns to predict the permutation index used to shuffle the patches.

Ex: The task is to place 9 shuffled image patches back in their original positions.

Algo:

- 1) randomly crop an image to 255×255 [red dashed window]
 - 2) Then divide into 3×3 grid cells, each with 75×75 size cell.
 - 3) A 64×64 tile is then randomly selected from this 75×75 cell.
A total of 9 tiles are extracted per image, which are then shuffled & used as input to ~~neural net~~ CNN with shared weights.
 - Reason for sampling 64×64 tile from 75×75 cell is to prevent model from learning trivial solutions. The model needs to learn the context of image.
 - if patches were exactly 75×75 model could learn to solve the puzzle by just matching the boundary pixels of the patches as adjacent patches would have shared a border.
 - 4) CNN processes each tile individually & produces a feature vector per patch index.
 - 5) Model is trained to place shuffled patches back into place.
 - achieved by training the model to predict the shuffled index used to generate shuffled tiles.
 - pretext task can be treated as classification.
 - cross entropy loss can be used as training objective.
- The task forces the n/w to become permutation invariant.
 - For 3×3 grid we have 64 permutations (classes)
 - can be a difficult task over the last pretext task.
 - proposed to follow pre-defined permutations to regulate task difficulty.

Pretext Task: Rotation

- model is trained to predict degree of rotation
- image x is randomly rotated at multiples of 90°

$$\text{Rot}(x, \phi); \phi \in \{0^\circ, 90^\circ, 180^\circ, 270^\circ\}$$

↳ results in a 4 class problem.

- context remains same by rotating the image.
model is forced to learn the same image context under different angles.
In this way, model has to learn object parts to conclude about the rotation.
- computationally cheap approach to define pretext task.
- All transformed samples pass through the same W_0 .

Learning:

- classification head used on backbone to learn rotation

- rotation task define as:

$$\mathcal{G} = \{g(x|y)\}_{y=1}^K$$

where $g(x|y) = \text{Rot}(x, (y-1)90^\circ)$ is rotation operator
 $y \in \{0, 1, 2, 3\}$

- classifier learns to predict y' , corresponding label of the rotation degree.

Output from the classifier is a prob. distr over all possible geometric transformations

$$f_\theta(x^y) = \{f_\theta^y(x^y)\}_{y=1}^K$$

$f_\theta^y(x^y) \rightarrow$ prob. distr. for geometric transformation

$\theta \rightarrow$ model's param.

- Empirically $K=4$ works well in practice.

- given a set of N training images, W_0 params are optimised to reduce rotation loss

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{\text{Rot}}(y, y')$$

$$\mathcal{L}_{\text{Rot}}(x_i, \theta) = \frac{1}{K} \sum_{y=1}^K \log(f_{\theta}^y(g(x_i)) | \theta)$$

- After training rotation prediction head is thrown away & backbone is used in the downstream task.

Pretext task: colorization

- model is trained to color grayscale images
- treated as pixel-wise prediction task.

each pixel is mapped to a quantised o/p color value in CIE LAB color space.

- problem can be treated as a regression task, where we can minimise diff. b/w regressed o/p & the ground truth color image.

However discretizing color space & treating prob. as classification prob. works better.

CE loss can then be employed.

Other pretext tasks

- image inpainting & content encoding
- clustering
- Auto Encoders can be used for unsupervised pretraining.
- other generic approaches (for non-image as well) which rely on contrastive learning exist.

04- SSL (Part-2) [PTO]

- 5 - Proxy tasks that are generic can be applied to any type of tasks/signals.

Generative modelling

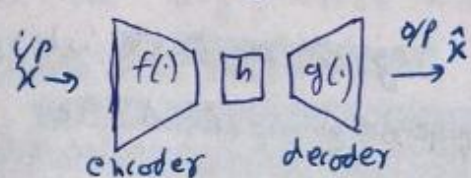
- 2 - formulate pretext task in the context of a generative model

Pretext-task: reconstruct the input while learning a meaningful representation

- in context of deep-NN, an AE performs this task.

AE is a NN trained to reconstruct its input.

it is composed of Encoder + Latent code + decoder



i/p: x ; o/p: $\hat{x}$

$$\text{loss } L = \|x - \hat{x}\|^2$$

$$\text{where } \hat{x} = g(f(x))$$

Denoising AE

i/p: it is partially corrupted by noise, coming from a prior distribution. This is a stochastic mapping of clean sig. x to a noisy sig. $\hat{x}$, represented by

$$\hat{x} \sim q_0(\hat{x}|x)$$

- The stochastic mapping $q_0(\cdot)$ can combine multiple types of prior distr.

o/p: given noisy sig. $\hat{x}$ as i/p, the o/p will be the clean sig.

$$\text{Loss: } L_{DAE} = \frac{1}{n} \sum_{i=1}^n (x^i - g(f(\hat{x}^i)))^2$$

- Training the denoising AE is the pretext task for self-supervised learning.

Contrastive Representation Learning

- goal is to learn an embedding space, i.e. a feature representation in which similar pairs of samples (ex. of same class) are close together and dissimilar pairs are far apart.

- can be applied to supervised/un-supervised learning.

- de-facto approach to perform self-supervision.

- developed in context of generative modelling.

- can be implemented with contrastive loss.

- 1st work was to learn similarity metric for face verification.

- Given an i/p x_i aim to **encode** it to an **embedding space** of d dimensions with mapping function:

$$f_\theta: \mathcal{X} \rightarrow \mathbb{R}^d$$
- **same class** samples with x_i should **lie close** in embedding space while samples of **diff. class** should **lie far apart** from x_i .
- **Contrastive loss** implements these 2 cond. into a **differentiable func.** that can be plugged into a deep neural network.
- given a pair of i/p (x_i, x_j) loss is:

$$\mathcal{L}_{\text{cont}}(x_i, x_j, \theta) = \mathbb{1}[y_i = y_j] \|f_\theta(x_i) - f_\theta(x_j)\|_2^2 + \mathbb{1}[y_i \neq y_j] \max(0, \epsilon - \|f_\theta(x_i) - f_\theta(x_j)\|_2)^2$$

$y_i \neq y_j \rightarrow$ corresponding labels of $x_i \neq x_j$

$\epsilon \rightarrow$ lower bound distance b/w samples of different classes.

$\hookrightarrow$ like a margin similar to SVM

$x_i \neq x_j \rightarrow$ are pair of samples/images of same class or diff. class.
 $\rightarrow$ both go through same NN f_θ .

- **+ve** pairs include samples/images of same class
- **-ve** " " " " " " " **diff** " "

- for each step of training sample x_i can be **compared with only one other** sample x_j of any class.
Loss is designed only for 1 to 1 comparison.

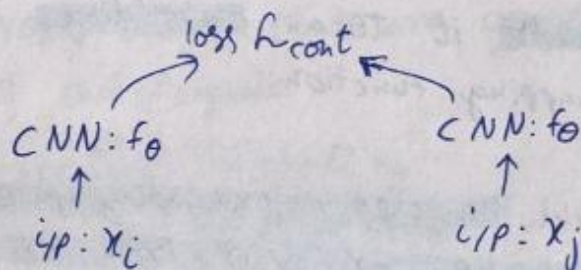
- **eventhough** current loss func. is calc. on a **single pair**, **many different pairs** can be generated for **1 i/p reference**.

- **No. of possible pairs** is much **larger** than no. of **individual images**. & each pair contributes to overall training signal of the model.

- the CNN used to implement contrastive loss is called **Siamese architecture**, because the i/p pair passes through same model with same parameters.

How can model be used for face recognition after contrastive loss training?

- match faces in the latent space to find a match.



Noise Contrastive Estimation (NCE) loss

- initially proposed to estimate parameters of unnormalized statistical models.
- applies non-linear logistic regression to differentiate b/w observed samples (+ve) & generated noisy samples (-ve)

$x \rightarrow$ target/observed sample

$$P(x | c=1; \theta) = p_\theta(x)$$

$\hookrightarrow$ data distr.

$\tilde{x} \rightarrow$ noisy sample

$$P(\tilde{x} | c=0; \theta) = q(\tilde{x})$$

$\hookrightarrow$ noise distr.

- observed samples refer to real data from a dataset

- noisy data is artificially created from predefined noise distribution

$c \rightarrow$ context vector/conditioning var.

NCE loss:

$$L_{\text{NCE}} = -\frac{1}{N} \sum_{i=1}^N [\log(\sigma(l_\theta(x_i))) + \log(1 - \sigma(l_\theta(\tilde{x}_i)))]$$

where
$$l_\theta(u) = \log \frac{p_\theta(u)}{q(u)}$$

$\sigma(l) \rightarrow$ sigmoid func. converts logits to probabilities

$\hookrightarrow$ this loss includes 1 +ve & 1 -ve sample.

How can we incorporate more samples in loss?

InfoNCE loss

- inspired by NCE loss.
- similar to N-pair loss, i.e. it includes multiple -ve samples in calculation of the contrastive loss.
- relies on the CE-loss to discriminate the +ve from a set of independent -ve samples.
- SOA contrastive learning approaches rely on InfoNCE like loss functions.

- given a context vector c , a +ve sample is drawn from the conditional distr. $P(x|c)$.

- for simplicity assume $X = \{x_i\}_{i=1}^N$ contains all samples with only 1 +ve sample.

Pror. of sampling a +ve sample

$$P(C = \text{pos} | X, c) = \frac{P(x_{\text{pos}} | c) \prod_{i=1 \dots N; i \neq \text{pos}} P(x_i)}{\sum_{j=1}^N P(x_j | c) \prod_{i=1 \dots N; i \neq j} P(x_i)} = \frac{f(x_{\text{pos}}, c)}{\sum_{j=1}^N f(x_j, c)}$$

- InfoNCE loss uses the categorical CE loss to identify one +ve sample among set of -ve samples.

$$\mathcal{L}_{\text{InfoNCE}} = - E_x \left[\log \frac{f(x, c)}{\sum_{x' \in X} f(x', c)} \right]$$

loss optimises -ve log likelihood of classifying +ve samples correctly.

soft-Nearest Neighbour loss

- considers multiple +ve samples.

- batch of samples $\{(x_i, y_i)\}_{i=1}^B$

$y_i \rightarrow$ label of x_i

$$\mathcal{L}_{\text{snh}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\sum_{j=1 \dots B} \exp(-f(x_i, x_j)/\tau)}{\sum_{k \neq i, j=1 \dots B} \exp(-f(x_i, x_k)/\tau)}$$

$f(\cdot, \cdot) \rightarrow$ function to measure similarity b/w 2 i/p.

$\tau \rightarrow$ temperature scaling param that tunes how well features are concentrated in feature space.

- low temperature allows low distance to dominate the loss, i.e. loss focuses more on low dist. compared to widely separated representations.

Can we work w/o +ve labels?

- contrastive learning requires +ve & -ve labels.

- label need not be object category.

- ex: each sample of dataset could be given a unique label & could use data augmentation to create multiple instances of it, for +ve label class. rest of dataset can form the -ve labels.

- True sample can be any data instance & its augmentations
 - -ve sample would be all other data instances & their augmentations
- Contrastive learning: pulls true samples & -ve samples individually closer to each other.
 pushes true & -ve samples away from each other.

What are challenges in defining true & -ve samples?

SSL challenges

- data augmentation should not ^{perturb} change semantics of true samples heavily in order to learn useful representation of the data.
- common cheap image augmentations:
Blurring, color jittering, horizontal flipping
- aug. could also ~~be~~ include rotation, translation, crop, cutmix and/or mixup
- SSL is based on backprop. & grad. descent based training.
 But no. of true/-ve samples within processed mini-batch affects the gradient updates.
- minimum mini-batch-size impacts many of the contrastive learning methods, in particular those that rely on -ve samples in the batch.
- Large mini-batch size includes large no. of -ve samples that are challenging for model to learn meaningful representations & distinguish b/w diff. samples.

What makes it challenging?

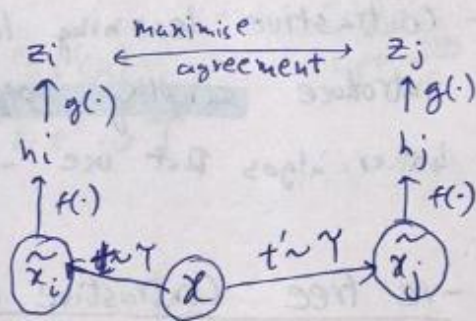
- Hard -ve mining

- Meaningful hard -ves are necessary to learn a robust feature representation. Increasing mini-batch size can help get hard-ves
- but, sampling w/o labels carries the risk of false -ve samples.
 Ex: true A-ve class include a dog.
 This is known as sampling bias.

Simple Contrastive Learning

(simCLR)

- simple framework for contrastive learning of visual representations is popular approach for contrastive learning.
- SimCLR algo. learns a visual representation by maximising agreement b/w differently augmented views of same sample via contrastive loss in the latent space.
- framework based on
 - Data augmentation
 - Neural n/w based encoder $f(\cdot)$
 - " " projection head $g(\cdot)$
 - Contrastive loss function



Algorithm

1. sample a mini-batch of size N
2. Each image x in the mini-batch is transformed twice
 $\tilde{x}_i = t(x)$; $t, t' \sim \gamma \rightarrow t, t'$ sampled from same augmentation γ
 $\tilde{x}_j = t'(x)$

After sampling augmentation effective mini-batch size is doubled i.e. $2N$.

3. Given one +ve pair & $(2N-1)$ -ve pairs, the representations are extracted using backbone n/w $f(\cdot)$
 $h_i = f(\tilde{x}_i)$ $h_j = f(\tilde{x}_j)$; $h_i, h_j \rightarrow$ raw representations of dimension D .

4. contrastive loss for each +ve pair is

$$L_{simCLR}^{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}[k \neq i] \exp(\text{sim}(z_i, z_k)/\tau)}$$

indicator func.
1 if $k \neq i$
0 if $k = i$

$\text{sim} \rightarrow$ cosine similarity score that operates on op of $g(\cdot)$

- $L_{simCLR}^{i,j}$ is just infoNCE loss with 1 pair of two samples & multiple pairs of -ve samples.

5. Use backbone from SimCLR pre-training & throw away $g(\cdot)$ in downstream task.

- empirically adding a projection head $g(\cdot)$ to transform raw rep. into lower dim. space $d < D$ improves contrastive learning perf.

$$z_i = g(h_i) \quad z_j = g(h_j)$$

$z \rightarrow$ projected feature onto lower dimensional head.

Limitations

- large mini-batch sizes consume memory & makes processing of each sample expensive.

- Performance is vulnerable to augmentation policy.

- Contrastive learning loss require -ve samples, which can introduce sampling error.

better algos that use -ve free contrastive loss exist.

-ve free Contrastive Learning

- Bootstrap Your Own Latent (BYOL) is a CL method that doesn't require -ve samples.

- Based on 2 neural n/w

- online n/w: learns to predict features extracted by target n/w from an i/p image under a different augmentation/view.

- Both n/w use same architecture

- Learned representation can be the finetuned to a downstream task

- similar to other SSL approaches.

BYOL

- online n/w parametrised by θ

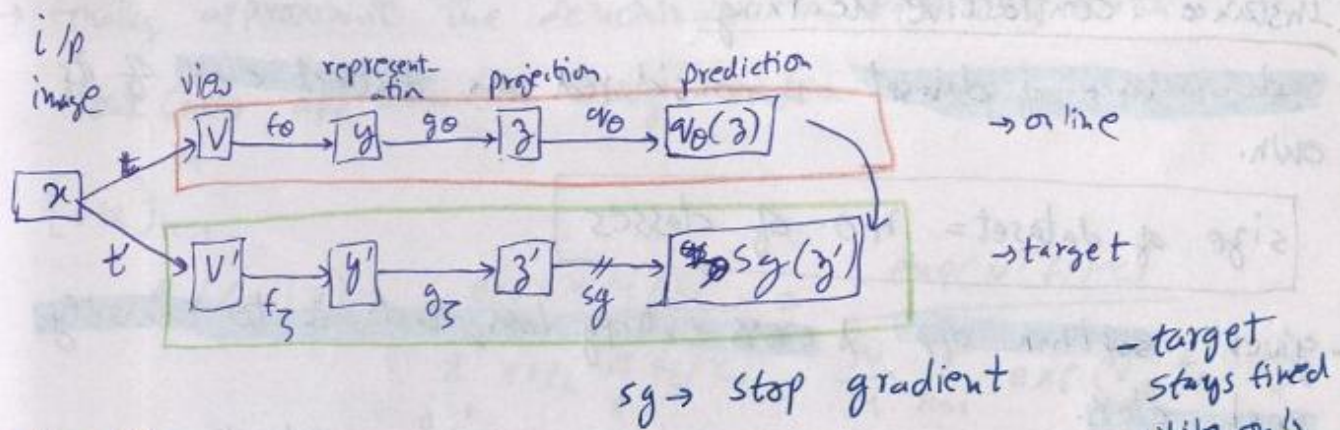
consists of encoder f_θ
projection head g_θ
predictor q_θ

$\rightarrow$ updated using grad descent

- target n/w parametrised by ζ

updated using exponential moving average.

$$\zeta \leftarrow \gamma \zeta + (1-\gamma) \theta$$



Algorithm

1. create 2 augmented views $v = t(x)$, $v' = t'(x)$
with augmentations sampled from $t, t' \sim \mathcal{T}$

2. Extract raw representations
 $y = f_0(v)$ $y' = f_z(v')$

3. Project into latent variables
 $z = g_0(y)$; $z' = g_z(y')$

4. online n/w o/p prediction $q_0(z)$.

Both $q_0(z)$ & z' are L2-normalized:

$$\bar{q}_0(z) = \frac{q_0(z)}{\|q_0(z)\|} \quad \bar{z}' = \frac{z'}{\|z'\|}$$

5. Calc. BYOL-loss (MSE)

$$L_{BYOL} = \|\bar{q}_0(z) - \bar{z}'\|_2^2$$

6. update online n/w with an optimiser ex: SGD
target " " exp moving avg. of 0.

- BYOL works as random sampling if batch normalisation is removed from the neural n/w.

- using -ve samples is important to prevent mode collapse.

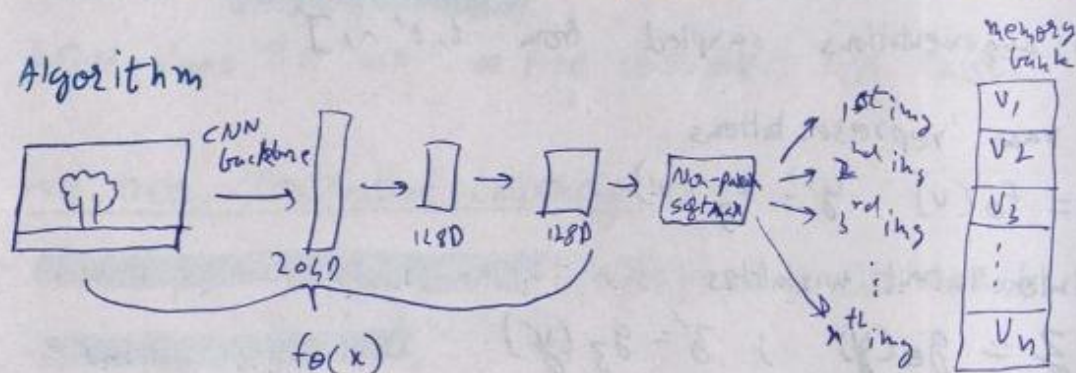
Instance contrastive learning

- each instance of dataset is considered as distinct class of its own.

size of dataset = n.o. of classes

- given softmax o/p & cross-entropy loss, task is to classify each class.
 - not possible to have a softmax with hundreds/millions of classes
 - hence use NCE loss on logit to approximate softmax o/p.

Algorithm



- $v = f_0(x) \rightarrow$ embedding function to learn

$\rightarrow$ vector v is normalised.

- non-parametric classifier used to predict probability of sample v belonging to class i

$$P(c=i|v) = \frac{\exp(v_i^T v / \tau)}{\sum_{j=1}^n \exp(v_j^T v / \tau)} ; \tau \rightarrow \text{temperature param.}$$

$\hookrightarrow$ denominator here needs access to representations of all samples which is an expensive computation.

$\hookrightarrow$ instead use a memory bank to store feature rep. from past iter. in a database instead of computing it every time.

$V = \{v_i\} \rightarrow$ memory bank

$f_i = f_0(x_i) \rightarrow$ features generated by forwarding the i/p image through network.

- $\rightarrow$ Rely on features stored in V instead of performing fwd pass every iter. to get f_i .

→ finally approximate the denominator in $P(c=i|v)$ with Monte Carlo approximation using random subset of M indices $\{j_k\}_{k=1}^M$

$$P(c=i|v) = \frac{\exp(v^T f_i / \tau)}{\sum_{j=1}^N \exp(v^T f_j / \tau)} = \frac{\exp(v^T f_i / \tau)}{\frac{N}{M} \sum_{k=1}^M \exp(v^T f_{j_k} / \tau)}$$

- since only 1 instance per class exists, training can be unstable, λ may severely fluctuate.

- To regularise training, additional term is introduced for the samples in the loss function. The final objective is

$$\mathcal{L}_{\text{instance}} = -\mathbb{E}_{p_d} [\log h(i, v_i^{t-1})] - \lambda \|v_i^t - v_i^{t-1}\|_2^2 - M \mathbb{E}_{p_n} [\log (1 - h(i, v_i^{t-1}))]$$

$$\text{where } h(i, v_i^{t-1}) = \frac{P(c=i|v)}{P(c=i|v) + M P_n(c=i)} \rightarrow \text{posterior prob. of sample } i \text{ being from the data distr.}$$

$v_i^{t-1} \rightarrow$ rep. stored from prev. iter.

noise distr is uniform $P_n = \frac{1}{N}$

- diff. $\|v_i^t - v_i^{t-1}\|$ diminish as learning embedding converges.

Momentum Contrastive Learning (MoCo)

- also relies on memory bank to store ^{feature} representations.

- memory bank is in form of a dynamic dictionary lookup, structured as a large FIFO queue of representations of all samples.

→ achieves faster access than instance contrastive learning.

Algorithm

1) given a query sample x_q , obtain query representation by using query encoder $q = f_q(x_q)$

- additionally a list of key representations $\{k_0, k_1, \dots\}$ extracted by momentum encoder are stored in dict. $k_i = f_k(x_i^k)$

- 2) Momentum encoder parametrised by Θ_k is progressively updated an Exponential Moving Average (EMA) of the encoder parametrised by Θ_q . Both n/w have same architecture

update step: $\Theta_k \leftarrow m\Theta_k + (1-m)\Theta_q$

- Assume there is one true k^+ in the dictionary that matches q . k^+ is created through a noisy (augmented) replicate of x_q .

Moco loss:
$$\mathcal{L}_{\text{Moco}} = -\log \frac{\exp(q \cdot k^+ / \tau)}{\sum_{i=1}^N \exp(q \cdot k_i / \tau)}$$

- Advantage ^{of Moco} over SimCLR: it decouples batch size from no of -ve samples by using a dictionary (memory bank).

$\therefore$ perf. not dependent on large batch sizes.

- Moco is diff. from instance discrimination through the FIFO structure that allows to use the representation of the immediately preceding mini-batch.

- Moco-v2 extends Moco using designs from SimCLR

↳ adds a projection head after encoder

↳ applies stronger data augmentation

↳ improves perf. of Moco

- In -

05 - Similarity learning

Supervised learning:

1) classification

- approximation function $f: \mathbb{R}^n \rightarrow 1 \dots K$ learns to map i/p sample to K categories, represented by a set of n dimensional features, to a categorical o/p.

- $y = f(x)$ is category prediction for i/p x where o/p can be probabilistic

- use Cross Entropy (CE) / Binary CE loss function.

- loss func. directly affects learnt feature representation.
- Ex: prediction of dog/cat from image

2) Regression

- mapping function $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ learns to predict numerical values.

- common loss function used: MSE

- learnt feature rep. is based on loss function.

- Ex: prediction of 2D keypoints on an image.

Similarity learning (Distance Metric learning)

- involves modelling similarities b/w 2 objects (ex: people & animals.)

- given a similarity function, images with similar context are projected into same neighbourhood in the latent space.

Images with different contexts are far apart in the latent space.

- Also known as distance metric learning.

- Deep NNs learn feature embeddings that fit a dist/similarity func.

Ex: euclidean dist can be used as loss func. for deep metric learning.

- learned rep. used to determine if 2 i/p samples belong to same class or not.

Deep Metric learning

- +ve/-ve samples missing here unlike in Contrastive learning.
(contrastive learning v/s metric learning)

- given: $\mathcal{X} \rightarrow$ training set

$\mathcal{Y} \rightarrow$ label set

goal: train a deep NN parametrised by θ producing a latent representation $f_{\theta}: \mathcal{X} \rightarrow \mathbb{R}^n$ of n dimensions based on the dist. func. $D: \mathbb{R}^n \rightarrow \mathbb{R}$

- For i/p samples $x_1, x_2 \in \mathcal{X}$ with labels $y_1, y_2 \in \mathcal{Y}$, dist. $D(f_{\theta}(x_1), f_{\theta}(x_2))$ has small value if $y_1 = y_2$ & large o.w.
- selection of dist. function can be done in advance & kept fixed during training.

Ex: Euclidean dist.

- NN architecture is important to the embedding learning.
- idea similar to contrastive learning.

Distance function:

- metric / dist. func. is a mathematical func. used to calc dist b/w pair of elements in a set.
- 2 main types of metrics
 - i) Pre-defined metrics
 - ii) Learned metrics
- Pre-defined metrics: metrics defined w/o knowledge of data
Ex: euclidean dist
- Learned metrics: metrics which can only be specified with the knowledge of the data.

Ex: Mahalanobis dist: $f(x-y) = (x-y)^T M (x-y)$;

$M \rightarrow$ matrix estimated from the data

- similarity measure is a task-relevant metric.

- common dist. functions:

numerical data: Euclidean dist., cosine similarity

categorical data: Hamming dist., Jaccard dist.

- There are different loss functions to learn a metric

common ones:

i) triplet loss (most popular)

ii) quadruplet loss

iii) Hierarchical triplet loss.

Triplet loss:

- $x_a, x_p, x_n \rightarrow$ triplet i/p samples from a given dataset

$y_a, y_p, y_n \rightarrow$ corresp. labels

such that $y_a = y_p$ and $y_a \neq y_n$

Ex: x_a & x_p are images of same person & x_n of another person.

$x_a \rightarrow$ anchor sample (query sample)

$x_p \rightarrow +ve$ "

$x_n \rightarrow -ve$ "

- goal of triplet loss: learn an embedding space where x_a & x_p are closer than x_a & x_n .

Ex: learn embeddings (latent rep.) of faces in database.

- overall, minimise dist. b/w x_a & x_p while dist. b/w x_a & x_n is maximised

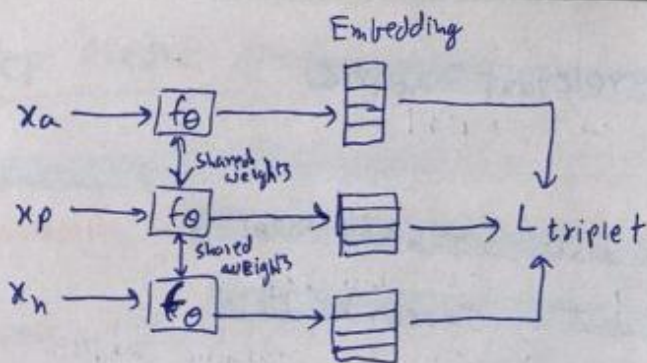
- loss function:

$$L_{\text{triplet}} = \max(0, D(f_0(x_a), f_0(x_p)) - D(f_0(x_a), f_0(x_n)) + \alpha)$$

$D \rightarrow$ euclidean dist. func.; $D = \|p - q\|_2^2$

$f_0 \rightarrow$ NN that extracts features

$\alpha \rightarrow$ margin that is enforced b/w +ve & -ve pairs.



- choose a difficult
- $f_\theta(\cdot)$ is normalised that dist. func. rec in range of $[0, 1]$.

- if $D(f_\theta(x_a), f_\theta(x_p)) \xrightarrow{\text{tends to}} 0$ * $D(f_\theta(x_a), f_\theta(x_n)) > D(f_\theta(x_p), f_\theta(x_n))$
 then x_n becomes an easy -ve.

- easy sample may not be useful for creating meaningful/large g
- learning robust rep. is based on selection of true & -ve samples.

How to select representative true/-ve samples? *

- 3 types of -ves Negative triplets

i) Easy triplets: have a loss of 0 because

$$D(f_\theta(x_a), f_\theta(x_p)) + \alpha < D(f_\theta(x_a), f_\theta(x_n))$$

ii) Hard triplets: x_n is closer to x_a than x_p

$$D(f_\theta(x_a), f_\theta(x_n)) < D(f_\theta(x_a), f_\theta(x_p))$$

iii) Semi-hard triplets: x_n is not closer to x_a than x_p but still have true loss

$$D(f_\theta(x_a), f_\theta(x_p)) < D(f_\theta(x_a), f_\theta(x_n)) + \alpha$$



- Std. approach: select a random semi-hard -ve for every pair of anchor & true samples.

Triplet mining

- forming triplets can be done online/offline.

• Offline triplets

- create triplets before starting the training.
- extract embeddings from f_θ [NN] for all training samples & then sample triplets.
- select only hard / semi-hard triplets for making mini-batches
- for all mini-batches, back-propagation & grad. descent are used to update the NN f_θ .
- computationally expensive as we use entire dataset to look for hard/semi-hard triplets.

In addition, each mini-batch formation is more expensive than std. supervised learning due to triplet computations.

Online triplets

- load samples on the mini-batch & compute triplets on the fly during training.
- not all triplets might be valid. We need 2 +ves and 1 -ve per triplet
- easy to implement & make training faster
- embeddings, gradients & param. updates performed in the same way with offline triplets.

steps

1. Fwd pass & embeddings
2. compute pairwise distances within the batch
3. Triplet candidate selection:
identify +ves & -ves

4. Apply mining from potential triplets (a, p, n)

5. Filter valid triplets: keep triplets with $D(a, p) > D(a, n)$
↳ model learns from mistakes

6. compute triplet loss

7. Back propagate & update

↳ can use hard-ve mining / semi-hard-ve mining instead of using every triplet too.

Challenges:

- n.o. of possible triplets $O(n^3)$
 - training with triplet loss can cost significant time & lead to inferior performance.
 - multiple correlated samples exist
 - training is highly redundant & less informative due to random sampling.
- ↳ optimisation can get stuck at local min due to these challenges
- other related loss functions Quadruplet loss & Hierarchical triplet loss offer faster convergence.

Quadruplet loss

- replaces Euclidean dist. with a learned metric
- learned dist. metric function $g(x_i, x_j)$ can model relationships b/w samples instead of a fixed metric.
- triplet loss is now:
$$L_{\text{triplet}} = \max(0, g(f_0(x_a), f_0(x_p))^2 - g(f_0(x_a), f_0(x_n))^2 + \alpha)$$

$g(x_i, x_j) \rightarrow$ represents dist b/w samples
high values correspond to dissimilar samples.
- The metric can be formulated as a FC layer followed by 2-dimensional o/p. (class-1 dissimilarity, class-2 similarity).
- one of these outputs can express dissimilarity probability of the 2 samples. Use it as ip to triplet loss.
- Triplet loss is based only on relative dist. b/w +ve & -ve pairs w.r.t anchor samples.
- Quadruplet loss imposes additional constraints by considering additionally a pair that is formed only from -ve samples.
i.e. the second -ve pair doesn't contain anchor image, instead only 2 -ve samples.

- Triplet loss doesn't consider variation of features $f(\cdot)$ while quadruplet loss is designed to encourage interclass variation, while encouraging smaller intraclass variation. i.e. it is designed to encourage both ^{larger} inter-class separation & tighter intra-class variation.

Samples: x_a, x_p, x_n, x_s

Labels: $y_a = y_p$

$y_a \neq y_n$

$y_a \neq y_s$

$y_s \neq y_n$

$x_a \rightarrow$ anchor sample

$x_p \rightarrow$ +ve sample

$x_n \rightarrow$ -ve sample

$x_s \rightarrow$ another -ve sample

- Quadruplet loss is:

$$L_{\text{quadruplet}} = \max(0, g(f(x_a), f(x_p))^2 - g(f(x_a), f(x_n))^2 + \alpha_1) + \max(0, g(f(x_a), f(x_p))^2 - g(f(x_s), f(x_n))^2 + \alpha_2)$$

$\alpha_1, \alpha_2 \rightarrow$ are 2 diff. margins

$$L_{\text{quadruplet}} = L_{\text{triplet}} + \max(0, g(f(x_a), f(x_p))^2 - g(f(x_s), f(x_n))^2 + \alpha_2)$$

$\hookrightarrow$ this 2nd term considers

dist. to be

dist. regardless of the anchor sample.

- 2nd term is auxiliary & thus it should not have same impact as the 1st one.

- The 1st term should have a large margin α_1 , as the anchor is used in both pairs. $\oplus$

- 2nd term can have small margin α_2 , because it is auxiliary term. $\oplus$

- The impact of the 2 terms is smoothed using 2 diff. margin thresholds according to the 2 points above. $\oplus$

This setup essentially uses 2 diff. weights to control influence of each term, making loss more flexible & effective, for training.

- margin affects the no. of hard samples to be used for training.
- margin threshold can't be too high, as it will produce too many hard samples & cause overfitting.
- adaptive margin threshold helps for selecting hard samples.
- Quadruplet loss computes the margin as the average dist. of the -ve & +ve sample distributions

$$a_{1,2} = w(\mu_n - \mu_p) = w \left(\frac{1}{N_n} \sum_{a,n} g(f(x_n), f(x_n)) - \frac{1}{N_p} \sum_{a,p} g(f(x_n), f(x_p)) \right)$$

$\mu_n, \mu_p \rightarrow$ means of -ve & +ve sample distr.

$N_n, N_p \rightarrow$ no. of -ve & +ve samples

$w = 1$ for a_1 & 0.5 for a_2

- computations performed on a mini-batch level
- Quadruplet loss originally proposed for person re-identification

Hierarchical Triplet loss

- is a variation where training set is organised into a hierarchy of classes.
- each class is associated with a set of child classes that are more specific than the parent class.

Hierarchical tree

- consider a deep NN f_θ that is pre-trained with std. triplet loss L_{triplet} .
- calc. dist. matrix of classes H , before constructing Re tree

$$d(p, q) = \frac{1}{n_p n_q} \sum_{i \in p, j \in q} \|f_\theta(x_i) - f_\theta(x_j)\|_2^2$$

$n_p, n_q \rightarrow$ cardinality of classes p & q .

- Use H to then construct hierarchy tree.
 - n.o. of tree leaves = n.o. of object classes.
each class = 1 leaf at level 0 (L0)
 - use H , grow tree recursively by merging leaf nodes at each level.
 - growth stops after reaching level L
 - avg inner dist. is used as threshold to merge nodes at level 0. defined as

$$d_0 = \frac{1}{C} \sum_{c=1}^C \frac{1}{n_c^2 - n_c} \sum_{i \in c, j \in c} \|f_0(x_i) - f_0(x_j)\|^2$$

$$n_c \rightarrow \text{n.o. of samples of class } c$$
 - at L^{th} level, threshold is defined based on d_0 as $d_L = \frac{1}{L} (4 - d_0)$ with L total levels.
 - 2 classes with dist smaller than d_L are merged to one node.
 - finally we have a tree with 1 top node & C leaf nodes

loss:

- 1) randomly select L' nodes at 0th level. ensures diversity in the mini-batch.
- 2) Based on dist. in feature space, $m-1$ nearest classes at 0th level of hierarchical tree are selected for each of the L' nodes.
 - done to learn features capturing diff. & similarities b/w visually similar classes.
- 3) t images per class are randomly collected.
 - total n.o. of images per mini-batch $\Rightarrow n = L' m t$
 - triplets are formed based on the collected n samples

- based on mini-batch collected triplets, hierarchical triplet loss is

$$L_M = \frac{1}{2Z_M} \sum_{T^z \in T^M} [\|x_a^z - x_p^z\| - \|x_a^z - x_n^z\| + a_z]$$

- $T^M \rightarrow$ mini-batch triplets

$$T^z = (x_a, x_p, x_n)$$

- $Z_M \rightarrow$ h.o. of triplets

$a_z \rightarrow$ dynamic range

$$a_z = \beta + d_H(y_a, y_n) - s_{y_a}$$

$\beta \rightarrow$ const. that encourages classes to be further apart than in prev. iterations

- $H(y_a, y_n) \rightarrow$ indicate tree level where 2 classes are merged into single node in next level

- $d_H(y_a, y_n) \rightarrow$ threshold for merging 2 classes.

$$s_{y_a} = \frac{1}{n_{y_a} - n_{y_n}} \sum_{i,j \in y_a} \|f_0(x_i) - f_0(x_j)\|^2 \rightarrow \text{avg dist b/w samples of class } y_a$$

- x_a encouraged to push nearby points with diff semantics far from itself.

it also contributes to the gradients of very distant samples by calculating dynamic margin.

06- Generative Models

- A generative model is a statistical model of the joint distr. of observed (i/p) & target (o/p) variables
 $P(x, y)$
- A discriminative model is a statistical model of conditional distr. of the target (o/p) given the observed (i/p) variables.
 $P(y|x)$
- Use Bayes rule to transform a generative model to a conditional o/p.
$$P(y|x) = \frac{P(x, y)}{P(x)}$$

Generative Adversarial Networks (GANs)

- Generative models are popular for sampling data, like images.
 - GANs popular for img. generation.
 - GANs align data & model distr.
- idea:
- Draw samples (x) from data distr. P_{data} , i.e. real images
 - Draw samples from model distr. (P_g) based on set of latent variables drawn from prior distr. P_z , i.e. generated synthetic images
 - Detect samples coming from the data distr. (real img) with the discriminator w .
- Learning Algo trains both discriminator (D) & generator (G) at the same time.
 - idea to have 2 w 's that compete with each other is called adversarial training.
 - model G generates images from i/p noise
 - " D classifies " into real/fake or generated.
 - " G tries to fool model D
 - " D acts as supervision on model G , hence we don't need labels to train the GAN.
 - At end of training keep the generator G .

- we need a real & a synthetic sample for a training iteration.

- The training has 2 main steps

1) Train discriminator:

- show both real & fake images to D-w/w

- trained to give high score (close to 1) for real img.

- low score (close to 0) for fake v.

2) Train generator:

- creates a batch of fake images $G(z)$

- goal is to make D believe its images are real.

- trained to update its param. s.t. D outputs a high score (close to 1) for its fake images.

- competition b/w 2 w/w drives both to improve until G-w/w can create realistic images

Generator's objective

$D(G(z)) \rightarrow 1$ w.r.t G param. update

generator's goal is to maximise Discriminator's score by updating its own param. (G param.)

Discriminator's objective

$D(G(z)) \rightarrow 0$ w.r.t D param. update

- job is to identify fake images

- updates its param. to make $D(G(z))$ closer to 0, which represents fake classification.

$D(x) \rightarrow 1$ w.r.t D param. update

↳ discriminator's objective when it sees a real image

↳ updates its param. to make o/r $D(x)$ closer to 1, which represents real classification.

Training process of GAN in K-training iter.

1) sample N noise samples from prior $p_z(z)$

$z \rightarrow$ random noise

2) sample N samples from real data distr.

3) update discriminator

4) sample N noise samples from prior $p_z(z)$

5) update generator (gradient based)

- idea of GAN is motivated by mini-max game
aim to minimize image reconstruction loss in G , while
maximizing perf. of discriminator to detect real & fake images.

Objective:

$$\min_G \max_D E_{x \sim p_{\text{data}}(x)} [\log(D(x))] + E_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

$p_{\text{data}}(x) \rightarrow$ data distr. over image x

$D(x) \rightarrow$ discriminator

$G(z) \rightarrow$ generator

$p_z(z) \rightarrow$ data distr over random noise z .

Advantages

- do not need a sampling method.
Using the gen. we can infinitely sample images.
- do not need to model interferences.
it's a feed-forward step
- can approximate wide range of func. Generate videos, text, audio...
- don't need labels for training GANs.

Autoencoder

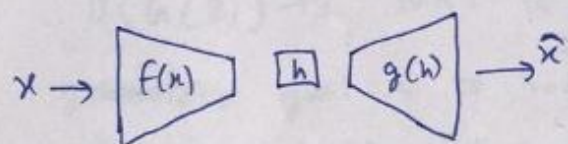
- is a NN that is trained to reconstruct its input.
composed of encoder, latent code & the decoder.
- AE tries to copy the input. Since latent code has smaller dimensions than the i/p, it can capture useful info in the latent code to reconstruct the i/p data.
- Encoder $f(\cdot)$ transforms the i/p x to latent code h , given by $h = f(x)$.
- Decoder $g(\cdot)$ reconstructs the i/p x from the latent code h given by $\hat{x} = g(h)$

i/p: x

o/p: $\hat{x}$

goal: $x \approx \hat{x}$

loss: $L = \|x - \hat{x}\|^2$ where $\hat{x} = g(f(x))$



- Learning to copy the i/p is known as recirculation.
- Motivation for AE was to reduce data dimensions.
- AE learns in an unsupervised manner.
- Unlike PCA, AE is a non-linear approach toward dimensionality reduction.

Undercomplete AE

- When latent code has smaller dimensions than the i/p data of an AE, it is called under-complete.
opposite is called over-complete.
- Learning a linear decoder is similar to spanning the subspace similar to PCA. However a non-linear dec. & enc. with right capacity is more powerful than PCA.
- Problem with AE is overfitting. Increasing capacity can cause memorizing data & not learning useful latent code.

- Use regularization approaches to overcome overfitting & constrain the latent code to learn useful info.

Regularized AE

- idea is to impose constraints during learning for extraction of useful info from i/p to latent code.

- constraints like:

- weight decay regularisation
- noise on i/p to become more robust (Denoising AE)
- small gradients during param. update (contractive AE)
- sparsity of latent code (sparse AE)

- AE can be formed with an MLP or CNN.

weight decay: Regularized AE

- uses std. L2 regularization

$$L_{RAE} = L + \beta \|w\|^2$$

$\beta \rightarrow$ controls influence of regularizer

$L \rightarrow$ std. MSE

Denoising ~~AE~~ AE

- AE learns the identity transformation, which can easily overfit. another approach to avoid this prob. is denoising AE.

- Motivation: human can recognise an image even when corrupted by noise. AE should be capable too.

- i/p: partially corrupted by noise from some prior distr. this is a stochastic mapping of clean sig. x to a noisy sig. $\hat{x}$:

$$\hat{x} \sim q_0(\hat{x}|x)$$

$q_0(\cdot) \rightarrow$ stochastic mapping can combine multiple types of prior distr.

In a simple case it can set no. of elements to 0.

- o/p: given noisy sig. $\hat{x}$, o/p is the clean signal.

- original model was shallow deep NN with only 1 hidden layer

- loss func. for n training samples is

$$L_{DAE} = \frac{1}{n} \sum_{i=1}^n (x^i - g(f(\hat{x}^i)))^2$$

↳ minimising this loss forces AE to extract features that contain useful info to reconstruct the clean signal.

- denoising AE can be interpreted as manifold learning where low-dimensional manifold is the latent code.

How to evaluate quality of latent code?

Stacked denoising AE

- extends orig. model to multiple hidden layers.

Learning is performed incrementally

- Train AE with single hidden layer. Corrupt i/p with noise.
- Add 2nd hidden layer
- Train 2nd hidden layer (only) by corrupting the 1st hidden layer o/p (only). i.e. we pass clean sig. from i/p & corrupt it after passing the 1st hidden layer.
- Add a 3rd hidden layer.
- Train 3rd hidden layer (only) by corrupting 2nd hidden layer o/p (only).
- ...

Variational Autoencoder (VAE)

VAE is a generative model to learn a latent (low-dimensional) rep. which is described in a probabilistic manner.

- std. AE learns a latent representation that is fixed for an input.
- VAE learns a prob. distr. for each value of the latent rep.
- Similar to AE, VAE also has 2 parts, enc. & decoder neural network.
- Instead of mapping i/p into a fixed vector, we aim to map it into a prob. distr. p_θ , parametrised by θ .

- goal: maximise likelihood of data x by the prob. distr.
 $p_0(x) = p(x|\theta)$.

- There is also a latent variable z to learn next to the θ .

- marginalise over z to find $p_0(x)$

$$p_0(x) = \int_z p_0(x, z) dz = \int_z p_0(x|z) p_0(z) dz$$

$p_0(x, z) \rightarrow$ joint distr. of x & latent rep. z

$p_0(x|z) \rightarrow$ likelihood

$p_0(z) \rightarrow$ prior over latent rep.

- integral of marginal likelihood $\rightarrow p_0(x)$ is intractable. $\rightarrow$ due to high dim. latent space
 $\rightarrow$ this is marginal likelihood

can't evaluate/differentiate $p_0(x)$

- true posterior $p_\theta(z|x) = \frac{p_0(x|z) p_0(z)}{p_0(x)}$ is also intractable.

$\hookrightarrow$ thus can't use EM algo or something related to compute it.
hence instead of calc. $p_0(x)$ directly introduce $q_\phi(z|x)$ & maximise ELBO

- introduce $q_\phi(z|x)$ to approximate posterior distr $p_\theta(z|x)$.

$$q_\phi(z|x) \approx p_\theta(z|x)$$

q is parametrised by ϕ

- in AE

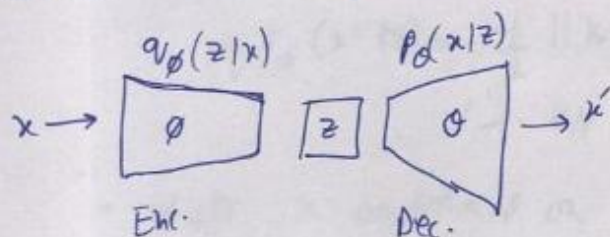
encoder approximates $q_\phi(z|x) \rightarrow$ posterior distr.

decoder computes $p_\theta(x|z) \rightarrow$ conditional distr.

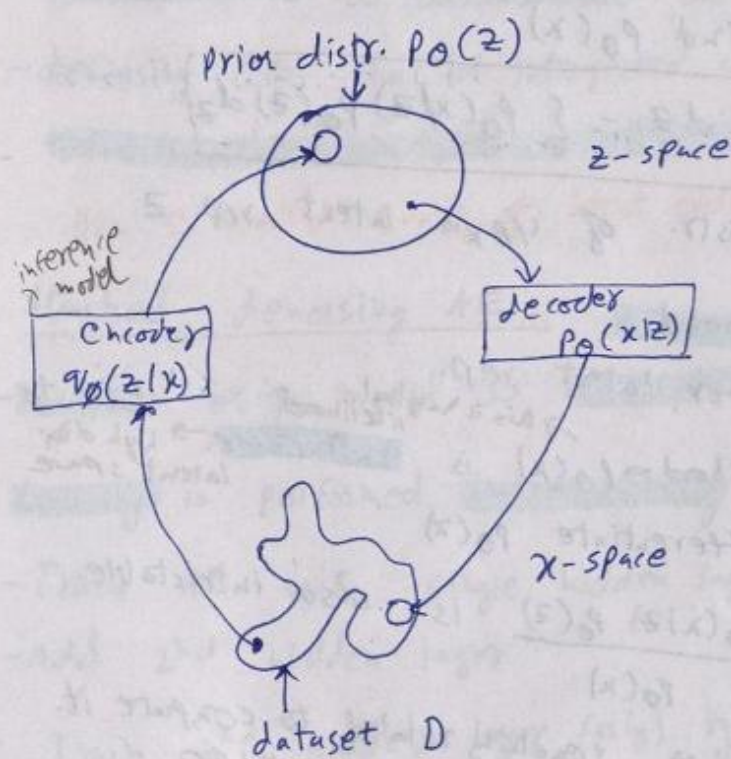
- VAE learns stochastic mappings b/w observed x -space & latent z -space.

- empirical distr. of x -space $q_0(x)$ is complex.

- learned distr. of latent z -space can be relatively simple.



- model learns joint distr $p_0(x, z)$, i.e. factorised as $p_0(x, z) = p_0(z) p_0(x|z)$ with a prior distr over latent space $p_0(z)$ & a stochastic decoder $p_0(x|z)$



- stochastic encoder $q_\phi(z|x)$, also called inference model approximates true but intractable posterior $p_0(z|x)$ of the model.

The goal of VAE is to max. marginal likelihood of data which is intractable.

To solve this VAE uses variational lower bound / ELBO.

ELBO is lower bound on log-likelihood of data & is used as loss func. to train VAE.

ELBO has 2 terms

- 1) reconstruction loss: measures how well the decoder can reconstruct original i/p from latent code.
- 2) KL div. term that regularises the learned latent distr. forcing it to be close to a predefined prior distr., typically a std. gaussian.

Variational Bound

given N samples as training set marginal likelihood is written as:

$$\log p_0(x^1, \dots, x^N) = \sum_{i=1}^N \log p_0(x^i)$$

$$\log p_0(x) = \underbrace{E_{q_\phi(z|x)} \left[\log \left[\frac{p_0(x, z)}{q_\phi(z|x)} \right] \right]}_{\text{variational lower bound}} + \underbrace{E_{q_\phi(z|x)} \left[\log \left[\frac{q_\phi(z|x)}{p_0(z|x)} \right] \right]}_{\text{non-ve KL divergence}}$$

$$\Rightarrow \log p_0(x^i) = \underbrace{D_{KL}(q_\phi(z|x^i) \parallel p_0(z|x^i))}_{\text{KL div. of approximated posterior from true posterior}} + \underbrace{L(\theta, \phi; x^i)}_{\substack{\text{variational lower} \\ \text{bound on marginal} \\ \text{likelihood} \\ \text{ELBO (Evidence} \\ \text{Lower Bound)}}}$$

- as KL-div. is non-ve, ELBO is a lower bound on log-likelihood of the data.

$$\Rightarrow \boxed{\log p_0(x^i) \geq L(\theta, \phi; x^i)}$$

$$\begin{aligned} \text{where } L(\theta, \phi; x^i) &= E_{q_\phi(z|x)} \left[\log \left[\frac{p_0(x, z)}{q_\phi(z|x)} \right] \right] \\ &= [-\log q_\phi(z|x) + \log p_0(x, z)] \end{aligned}$$

hence for sample i

$$\boxed{L(\theta, \phi; x^i) = \underbrace{-D_{KL}(q_\phi(z|x^i) \parallel p_0(z))}_{\text{regularisation term}} + \underbrace{E_{q_\phi(z|x^i)} \log p_0(x^i, z)}_{\text{reconstruction loss}}}$$

→ we want to use ~~ELBO~~ variational lower bound on marginal likelihood of each sample x^i as the loss function to train VAE.

$\log p_0(x^i|z)$ is implemented as reconstruction loss

$$\log p_0(x^i|z) = -\frac{1}{2} \|x - x'\|_2^2$$

$x' \rightarrow$ o/p of decoder

- distr. x conditioned on z is modelled as gaussian centered on decoder o/p. - $q_\phi(z|x^i)$ & $p_0(z)$ are also chosen as gaussian

VAE computational flow

1. Encoder/inference model

takes i/p data point x maps it to param. of gaussian in latent space.

2. Reparametrisation trick

- used as directly sampling from a distr. is a non-differentiable process, making back-prop. to train encoder impossible.

3. Decoder

Instead of sampling from distr., a sample from latent space denoted as z , it is generated as a deterministic func. of μ & σ from encoder & independent random sample of gaussian noise ϵ

$$z = \mu + \epsilon \odot \sigma$$

3. Decoder

- takes latent vector z & reconstructs new data sample x' .

- this n/w is trained to maximise likelihood of reconstructed o/p, effectively learning how to generate realistic data from the latent space.

ELBO optimisation

- optimise

ϕ (variational parameters) for encoder

θ (generative parameters) for decoder

using gradient descent.

- given training dataset, ELBO obj. is sum/avg of individual data points

$$L_{\theta, \phi}(D) = \sum_{x \in D} L_{\theta, \phi}(x)$$

- gradients of ELBO WRT θ are easy to obtain

$$\nabla_{\theta} L_{\theta, \phi}(x) = \nabla_{\theta} E_{q_{\phi}(z|x)} [\log p_{\theta}(x, z) - \log q_{\phi}(z|x)]$$

$$= E_{q_{\phi}(z|x)} [\nabla_{\theta} (\log p_{\theta}(x, z) - \log q_{\phi}(z|x))] \quad \text{if } \phi \text{ is fixed}$$

$$\approx \nabla_{\theta} (\log p_{\theta}(x, z) - \log q_{\phi}(z|x)) = \nabla_{\theta} (\log p_{\theta}(x, z))$$

- grad. WRT ϕ are difficult to obtain, as we can't put ∇_{ϕ} inside the expectation.

$$\nabla_{\phi} L_{\theta, \phi}(x) = \nabla_{\phi} E_{q_{\phi}(z|x)} [\log p_{\theta}(x, z) - \log q_{\phi}(z|x)]$$

$$\neq E_{q_{\phi}(z|x)} [\nabla_{\phi} (\log p_{\theta}(x, z) - \log q_{\phi}(z|x))] \quad \text{because } \nabla_{\phi} \text{ is outside}$$

Hence we have an undifferentiable expectation.

Lemma: general example that also applies to VAE

$E_{p(z)} [f_{\theta}(z)] \rightarrow$ consider this expectation

$p(\cdot)$ is a density function

$f_{\theta}(\cdot)$ is differentiable

can easily compute grad. ∇_{θ}

$$\begin{aligned} \nabla_{\theta} E_{p(z)} [f_{\theta}(z)] &= \nabla_{\theta} \left[\int_{\mathbb{Z}} p(z) f_{\theta}(z) dz \right] = \int_{\mathbb{Z}} p(z) [\nabla_{\theta} f_{\theta}(z)] dz \\ &= E_{p(z)} [\nabla_{\theta} f_{\theta}(z)] \end{aligned}$$

hence grad. of expectation = expectation of gradient //

Now consider the foll. expectation

$E_{p_{\theta}(z)} [f_{\theta}(z)]$; $p_{\theta}(z) \rightarrow$ density func. also parametrised by θ .

computing the grad.

$$\begin{aligned} \nabla_{\theta} E_{p_{\theta}(z)} [f_{\theta}(z)] &= \nabla_{\theta} \left[\int_{\mathbb{Z}} p_{\theta}(z) f_{\theta}(z) dz \right] \\ &= \int_{\mathbb{Z}} \nabla_{\theta} [p_{\theta}(z) f_{\theta}(z)] dz = \int_{\mathbb{Z}} \nabla_{\theta} p_{\theta}(z) f_{\theta}(z) dz + \int_{\mathbb{Z}} p_{\theta}(z) \nabla_{\theta} f_{\theta}(z) dz \\ &= \int_{\mathbb{Z}} \nabla_{\theta} p_{\theta}(z) f_{\theta}(z) dz + E_{p_{\theta}(z)} [\nabla_{\theta} f_{\theta}(z)] \end{aligned}$$

Problem: can't take a gradient

- use reparametrisation trick to get gradients.

Reparametrisation trick

- ELBO can be easily differentiated w.r.t. θ & ϕ by changing variables.

- express random variable (RV) $z \sim q_{\phi}(z|x)$ as a differentiable transformation of another (RV) ϵ , given $z \propto \phi$.

$$z = g(\epsilon, \phi, x)$$

$\epsilon \sim p(\epsilon) \rightarrow$ gaussian distr. with 0 mean & variance 1

ϵ is independent of x & ϕ .

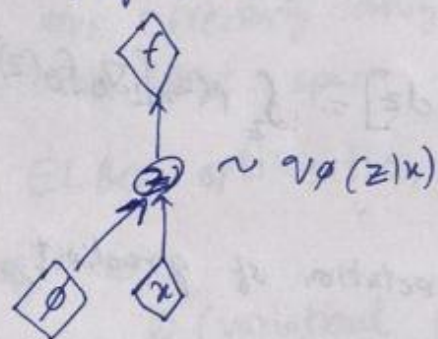
- consider overall objective as $f(\cdot)$, expectation is

$$E_{q_{\phi}(z|x)}[f(x)] = E_{p(\epsilon)}[f(g(\epsilon, \phi, x))]$$

then grad. is

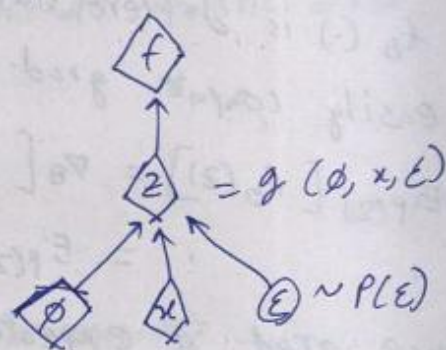
$$\begin{aligned} \nabla_{\phi} E_{q_{\phi}(z|x)}[f(z)] &= \nabla_{\phi} E_{p(\epsilon)}[f(g(\epsilon, \phi, x))] \\ &= E_{p(\epsilon)}[\nabla_{\phi} f(g(\epsilon, \phi, x))] = \nabla_{\phi} f(g(\epsilon, \phi, x)) \end{aligned}$$

orig. form



$\hookrightarrow$ orig. form, we can't backprop. through RV z .

Reparametrised form



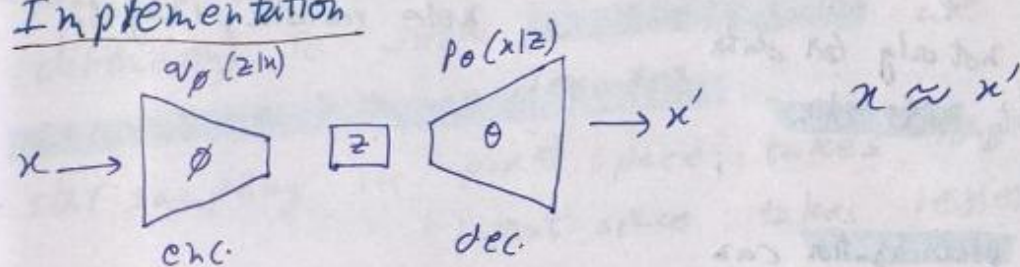
$\hookrightarrow$ Reparametrize z as deterministic & diff func. of ϵ, ϕ, x .

- After reparametrisation ELBO is

$$\begin{aligned} \mathcal{L}_{\theta, \phi}(x) &= E_{q_{\phi}(z|x)}[\log p_{\theta}(x, z) - \log q_{\phi}(z|x)] = \\ &= E_{p(\epsilon)}[\log p_{\theta}(x, z) - \log q_{\phi}(z|x)] \\ &\quad \text{with } z = g(\epsilon, \phi, x) \end{aligned}$$

- given $g(\cdot)$ is differentiable, can use MC (Monte Carlo) estimator to estimate gradients.

Implementation



encoder: approximated **posterior distr.** $q_{\phi}(z|x)$

decoder: conditional **likelihood distr.** $p_{\theta}(x|z)$

orig. loss: $L(\theta, \phi, x^i) = -D_{KL}(q_{\phi}(z|x^i) \parallel p_{\theta}(z)) + \mathbb{E}_{q_{\phi}(z|x^i)} [\log p_{\theta}(x^i|z)]$

$p_{\theta}(z)$ & o/p of enc. are fixed & modelled as gaussian.

- use reparametrization trick to sample z
- finally condition on z to generate x .

Training:

- 1) Provide x to enc. to get μ_x & σ_x
- 2) sample gaussian noise $\epsilon \sim \mathcal{N}(0, 1)$ with $0 \rightarrow \mu$ & $1 \rightarrow \sigma$
- 3) reparametrize $z = \mu_x + \epsilon \sigma_x$
- 4) Provide z to decoder & get generated sample x' .
- 5) compute loss functions

reconstruction loss: $\frac{1}{2} \|x - x'\|_2^2$

variational loss: $-D_{KL}(\mathcal{N}(\mu_x, \sigma_x) \parallel \mathcal{N}(0, 1))$

Total loss: $\frac{1}{2} \|x - x'\|_2^2 - D_{KL}(\mathcal{N}(\mu_x, \sigma_x) \parallel \mathcal{N}(0, 1))$

- 6) compute gradients, back propagate & update parameters of encoder & decoder.

Inference:

- trained VAE used to sample x
- given trained model one needs only learned μ_x & σ_x & decoder to generate samples.

Adv

- learning smooth latent space representation of i/p data
- can be used not only for data generation but also dim. reduction.
- latent space visualization can be beneficial for other tasks.

Disadvantage

- blurry & unrealistic o/p. GAN produces sharper & more realistic images.

07- Generative Models - II

- we look at generative models where no labels are provided like in unsupervised learning & self-learning.

training set $D = \{x_1, \dots, x_n\}$

↳ each sample can be an image.

- generative model $p_\theta(x)$ learns to form images based on training set D , which expresses the distribution $P_{data}(x)$
- generative model can be used for sampling as it learns the data distr. based on the training set.

Generative Model training

- 2 approaches to train generative model $p_\theta(x)$ to match the data distr. $P_{data}(x)$

1) Max. expected log-likelihood

2) Min. divergence b/w $p_\theta(x)$ & $P_{data}(x)$

- GANs minimise Jensen-Shannon divergence: $\min_{\theta} \max_{\phi} E_{x \sim P_{data}(x)} [\log p_\theta(x)] + E_{z \sim p_\phi(z)} [\log (1 - p_\theta(z))]$

- VAEs max log-likelihood of ELBO

$$L(\theta, \phi; x_i) = -D_{KL}(q_\phi(z|x_i) || p_\theta(z)) + E_{q_\phi(z|x_i)} [\log p_\theta(x_i)]$$

Gen. model sampling

- Both GANs & VAEs sample noise z (based on prior distr) in latent space to create a new sample in 1 step.
- Diffusion models starts with fully noising sample & gradually denoise it into an image. [have to use NN many times]

where is randomness? $\rightarrow$ we start with noise each time.

- we deal with noise on the sample space ex: img. space
- possible to have noise in latent space as well (latent diffusion). To work in latent space we need an encoder & then a decoder.
- star sampling in pixel space: takes longer
latent space: takes lesser time

Diffusion Models

- Motivation: start from base distribution, like Gaussian distr. & gradually convert it into target distr. like an image.
- process is like a Markov chain. Each step in transition from noise - img. is a step in the chain.
- assumption: current step depends only on the previous.
- Due to Markov assumption, we can define the diffusion process as seq. of gaussian transitions.
In end we will have a Gaussian distr. given a large no. of steps.
- These models have step-wise functionality not only during sampling but also during training.
- diffusion model consists of

Fwd process (img-noise) $\rightarrow$ training
Reverse " (noise-img)

Sampling (noise-img) $\rightarrow$ inference.

Forward Process

- gradually add small amounts of Gaussian noise to the image.
each step makes img x_t noisier until we end up with noise x_T .
- adding noise is ~~repeated~~ repeated for T steps.
- end up with pure noise (isotropic Gaussian distr.) after large no. of iterations.

illustration of process: $(x_0) \rightarrow \dots \rightarrow (x_{t-1}) \rightarrow (x_t) \rightarrow \dots \rightarrow (x_T)$
 $q(x_t | x_{t-1}) \rightarrow$ diff. process approximation

- image noising at each time step

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon$$

$\epsilon \sim \mathcal{N}(0, I)$ & β_t the noise variance

Noise variance $\beta_1 \dots \beta_T$ is from 0 to 1, Tells us amount of noise to add at the current step.

- At each step noise with more & more variance is added to the image.

- based on update step x_t , sampling as a Gaussian distr. is

$$q(x_{t+1} | x_t) = \mathcal{N}(\sqrt{1 - \beta_t} x_t, \beta_t I)$$

$\beta_t \rightarrow$ current variance schedule

$\sqrt{1 - \beta_t} x_t \rightarrow$ mean value

- we can express x_t WRT x_0 as

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\alpha_t} x_0, (1 - \alpha_t) I)$$

$$\text{where } \alpha_t = \prod_{i=1}^t \alpha_i$$

$$\text{Also } q(x_{t+1} | x_t) = \mathcal{N}(\sqrt{\alpha_t} x_t, 1 - \alpha_t I)$$

$$\text{with } \alpha_t = 1 - \beta_t$$

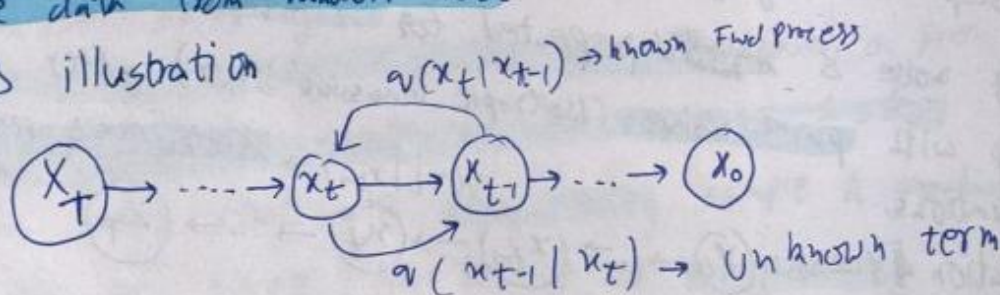
Reverse Process

- noising step can't help create an image, but opposite approach (denoising) can form an image from Gaussian noise in several steps. $\rightarrow$ reverse process

goal! learn with the diffusion model to go from the noise to image using a multiple step process. $q(x_{t-1} | x_t)$

- generate data from random noise.

Process illustration



- reverse process is unknown $q(x_{t-1}|x_t)$

consider Bayes' rule

$$f(a|b) = \frac{f(a,b)}{f(b)} = \frac{f(a) f(b|a)}{f(b)}$$

$$\Rightarrow q(x_{t-1}|x_t) = \frac{q(x_{t-1}, x_t)}{q(x_t)} = \frac{q(x_{t-1}) q(x_t|x_{t-1})}{q(x_t)}$$

$$= q(x_t|x_{t-1}) \frac{q(x_{t-1})}{q(x_t)}$$

$q(x_t) \rightarrow$ intractable computationally as $q(x_t) = \int q(x_t|x_{t-1}) q(x_{t-1}) dx_{t-1}$

$q(x_t|x_{t-1}) \rightarrow$ gaussian distr. which is known

- hence rely on some approximation to compute $q(x_{t-1}|x_t)$

- assumption: FWD process ^{steps} adds ^{very} small noise with noise schedule β_t .

$\rightarrow$ based on this we can assume rev. process can be a

gaussian distr. similar to fwd. process.

- hence rev. process approximation is a gaussian with unknown mean & variance.

- rely on a NN to learn these unknown param. for the reverse process.

- NN model is defined as

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t); \Sigma_\theta(x_t, t))$$

goal in training: minimise the log-likelihood of generated data using the reverse process.

- objective similar to VAE's training.

$$L(\theta, \varphi; x^i) = -D_{KL}(q_\varphi(z|x^i) \| p_\theta(z)) + \mathbb{E}_{q_\varphi(z|x^i)} [\log p_\theta(x^i|z)]$$

$$\mathbb{E}_{q_\varphi(z|x^i)} [\log p_\theta(x^i|z)] - D_{KL}(q_\varphi(z|x^i) \| p_\theta(z)) \leq \log p_\theta(x^i)$$

$\rightarrow$ use same objective for diffusion as well

Learning the reverse process

- we want to learn $p_\theta(x_{t+1}|x_t) \rightarrow$ rev. process; from available fwd process $q(x_t|x_{t+1})$

- objective func: KL divergence with all time steps T

$$D_{KL}(q(x_{1:T}|x_0) || p_\theta(x_{1:T}|x_0))$$

- similar to VAEs, apply variational inference & optimise ELBO on likelihood $p_\theta(x_0)$ of sample x_0 .

$$\begin{aligned}\log p_\theta(x_0) &\geq \log p_\theta(x_0) - D_{KL}(q(x_{1:T}|x_0) || p_\theta(x_{1:T}|x_0)) \\ &= \log p_\theta(x_0) - E_{x_{1:T} \sim q(x_{1:T}|x_0)} \left[\frac{\log q(x_{1:T}|x_0)}{p_\theta(x_{1:T}|x_0)} \right] \\ &= \log p_\theta(x_0) - E_q \left[\frac{\log q(x_{1:T}|x_0)}{p_\theta(x_{0:T})} + \log p_\theta(x_0) \right] \\ &= -E_q \left[\frac{\log q(x_{1:T}|x_0)}{p_\theta(x_{0:T})} \right]\end{aligned}$$

hence $\boxed{\log p_\theta(x_0) \geq -E_q \left[\frac{\log q(x_{1:T}|x_0)}{p_\theta(x_{0:T})} \right]}$

- Next tx. ELBO into sum of simpler terms

Consider variational lower bound loss $L_{VLB} = E_q \left[\frac{\log q(x_{1:T}|x_0)}{p_\theta(x_{0:T})} \right]$

$$\begin{aligned}&= E_q \left[\log \frac{\prod_{t=1}^T q(x_t|x_{t-1})}{p_\theta(x_T) \prod_{t=1}^T p_\theta(x_{t-1}|x_t)} \right] = \dots = E_q \left[D_{KL}(q(x_T|x_0) || p_\theta(x_T)) \right] \\ &\quad + \sum_{t=1}^T D_{KL}(q(x_{t+1}|x_t, x_0) || p_\theta(x_{t+1}|x_t)) \\ &\quad - \log p_\theta(x_0|x_1)\end{aligned}$$

with 3 terms we write it as

$$L_{VLB} = \underline{L_T} + \underline{L_{T-1}} + \dots + \underline{L_0}$$

$L_T \rightarrow$ prior loss & compares final x_T with x_0

$L_0 \rightarrow$ reconstruction term, prob. of true x_0 given current prediction x_1

Diffusion loss is $L_t = D_{KL}(q(x_{t+1}|x_t, x_0) || p_\theta(x_{t+1}|x_t))$

$$= D_{KL}(q(x_t | x_{t+1}, x_0) || p_\theta(x_t | x_{t+1})) \text{ for } 1 \leq t \leq T-1$$

↳ measures similarity of learned rev. process

$p_\theta(x_t | x_{t+1})$ with real rev. process

$q(x_t | x_{t+1}, x_0)$.

this part of L_{VLB} is imp.

Diffusion loss

- we know $q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{a}_t} x_0, (1 - \bar{a}_t) I)$ with $\bar{a}_t = \prod_{i=1}^t \alpha_i$

$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{a_t} x_{t-1}, (1 - a_t) I)$ with $a_t = 1 - \beta_t$

- using Bayes' rule

$q(x_t | x_t, x_0) = \mathcal{N}(\tilde{\mu}(x_t, x_0), \tilde{\beta} I)$ for some $\tilde{\mu} \propto \tilde{\beta}$.

also assume

$p(x_{t+1} | x_t) = \mathcal{N}(\mu_\theta(x_t), \tilde{\beta}_t I)$

$\mu_\theta(\cdot) \rightarrow$ Neural Network.

- based on $\mu_\theta(x_t) \propto \tilde{\mu}(x_t, x_0)$ diff. loss becomes to minimise their diff. while assuming same variance $\tilde{\beta}_t$.

- the term $q(x_t | x_t, x_0) = \mathcal{N}(\tilde{\mu}(x_t, x_0), \tilde{\beta} I)$ is further worked by considering $\tilde{\mu}(x_t, x_0)$

$\tilde{\mu}_t = \frac{1}{\sqrt{a_t}} \left(x_t - \frac{1 - a_t}{\sqrt{1 - \bar{a}_t}} \epsilon_t \right)$; $\epsilon_t \rightarrow$ noise added to x_0 resulting in x_t .

- model now becomes

$\mu_\theta(x_t, t) = \frac{1}{\sqrt{a_t}} \left(x_t - \frac{1 - a_t}{\sqrt{1 - \bar{a}_t}} \epsilon_\theta(x_t, t) \right)$; $\epsilon_\theta(x_t, t) \rightarrow$ NN model of noise ϵ_t .

- with this model a parametrisation, minimising the diff. loss L_t refers to predicting the noise ϵ_t using NN model $\epsilon_\theta(\cdot)$ & then removing it from x_t .

- Finally diffusion loss L_t is

$$L_t = E_{x_0, \epsilon} [c_1 \cdot \|\tilde{\mu}(x_t, x_0) - \mu_\theta(x_t, t)\|^2] =$$

$$E_{x_0, \epsilon} [c_2 \cdot \|\epsilon_t - \epsilon_\theta(x_t, t)\|^2]$$

$$= E_{x_0, \epsilon} [c_2 \cdot \|\epsilon_t - \epsilon_\theta(\sqrt{a_t}x_0 + \sqrt{1-a_t}\epsilon_t, t)\|^2]$$

$$\text{with } x_t = \sqrt{a_t}x_0 + \sqrt{1-a_t}\epsilon_t$$

c_1 & c_2 are two const we can ignore

- optimising diff. loss is

1) sample x_0

2) sample a time step $t, \dots, T$

3) sample noise $\epsilon_t \sim \mathcal{N}(0, I)$

4) generate noisy $x_t = \sqrt{a_t}x_0 + \sqrt{1-a_t}\epsilon_t$

5) compute loss $L_t = \|\epsilon_t - \epsilon_\theta(x_t, t)\|^2$

6) perform gradient descent on L_t

train until convergence.

Diffusion loss interpretation

- During forward process: ~~can~~ can randomly pick up a time step & add noise ϵ_t to the noisy sample (image)

- Reverse process: take noisy sample x_t & denoise it using the NN $\epsilon_\theta(\cdot)$. But, NN doesn't remove directly noise from sample, instead it produces the noise that should be removed from the sample.

- Diff. models are commonly used to generate img, video & graphs

- most common architecture is to rely on a U-Net style architecture

Comparison to other generative models

- 2 main metrics

1) FID (Frechet Inception Distance):

calculates statistical diff. b/w real & generated images' features representations extracted from pre-trained Inception-V3 model.

lower FID score = better quality & diversity of generated images compared to real ones.

lower FID $\rightarrow$ better

2) IS (Inception Score)

Measures quality by classification confidence & measures the diversity, by spread of predicted classes across generated images.

higher IS = better

- based on std. bench marks

Diff models $>$ GANs $>$ VAEs

Latent Diffusion Models (LDMs)

- Pixel space diffusion models have drawbacks

- sampling speed is very slow

- training on high-resolution images is resource intensive.

- LDMs fix this issue with

1) Encode everything into latent space with help of pre-trained VQGAN encoder E .

2) perform diffusion in latent space of VQGAN.

$\rightarrow$ diffusion is performed in lower dimension

3) Decode back to pixel space with the VQGAN decoder D .

sampling speed is still problematic.

- Sol: change sampling strategy. Trade sampling speed for quality with Denoising Diffusion Implicit Models (DDIM)

Conditional sampling & Guidance

- condition sample generation on additional info.

ex: image generation conditioned on a sentence.

08- Model Compression

Demands of a NN:

- performance of NN increases with the number of model parameters.
- Re increase leads to increased
 - Memory & computing resources
 - Execution time
 - Energy consumption
- Real-time is often only possible with high-performance workstations.

Model compression

- simplifies the model in terms of parameter number and/or parameter size, while aiming not to reduce the model performance.

Lossless model compression retains model performance.

Lossy " " reduces " " " "

- In some cases model performance can be improved after model compression. Why?

Compression methodologies

- Parameter pruning: deals with removal of redundant parameters.
- Parameter quantisation: maps large (or infinite) set of continuous values to a smaller set of discrete finite values.
- Knowledge distillation: compresses large model into a smaller model.
- Low rank approximation / factorization: approximates redundant parameters by a linear combination of smaller set of parameters.
- Neural architecture search: finds model architecture that minimises performance ~~of~~^{on} the training set.

Parameter Quantisation

- aim is to reduce precision of the weights of the model.

Ex: reducing precision of weights from 32-bit floating point to 8-bit integer lowers the computational effort & memory demand of the model by a factor of 4.

- but, quantisation is an irreversible process.

Quantised 8-bit model can't be converted back to 32-bit model due to loss of information.

Quantisation Precision

- NN can be quantised to diff. precision

precision = bit width of quantised model weights

- common for NN to be trained & executed with 32 bit floating point precision.

They then can be pruned from float32 to different levels of precision.

- common approaches:

→ 16-bit floating point: doesn't reduce performance of the model

→ 8-bit int: perf. loss is unavoidable

→ 1 bit: weights of binary NN have 1 bit precision

→ Mixed precision: layers in the NN have different precision regarding their influence on the o/p.

Categories of Quantisation methods

- Uniform & non-uniform quantisation

- symmetric & asymmetric quantisation

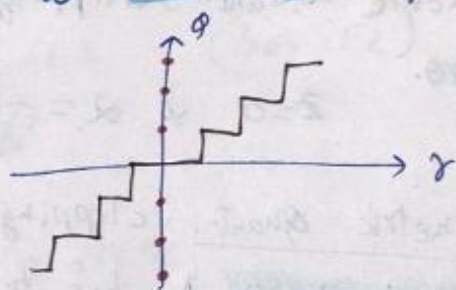
- Layerwise & channel-wise quantisation

- Based on protocol: quantisation before, during or after training.

→ Data free Quant. doesn't require access to training data.

Uniform & non-uniform Quantisation

- real cont. values r are mapped to discrete lower precision domain Q . (.) *on graph*



- uniform quantisation has distance between red dots (i.e. quantisation levels) to be the same.

$$Q(r) = \text{Int} \left(\frac{r}{s} \right) - z$$

$s \rightarrow$ real valued scaling factor

$z \rightarrow$ integer zero point

$$s = \frac{\beta - \alpha}{2^b - 1}$$

$[\alpha, \beta] \rightarrow$ Clipping range

$b \rightarrow$ bit width

- For non-uniform quantisation, dist. b/w red dots can vary.

Example:

- Q. Consider conversion of 32-bit float with value of $r = 0.5$ to an 8-bit float with a clipping range of $[-1, 1]$ & integer zero point of $z = 0$.

- here $b = 8$; $\alpha = -1$, $\beta = 1$

$$s = \frac{1 - (-1)}{2^8 - 1} = \frac{2}{255} \approx 0.0078$$

$$Q(r) = \text{Int} \left(\frac{0.5}{0.0078} \right) - 0 = \text{Int}(64.1) = 64$$

hence float 32 with value 0.5 will be quantised to a value of 64 in 8-bit.

Symmetric Quantisation

- symmetric Quant: clipping range is equally distributed around zero.

$$z=0 \quad \& \quad \alpha = -\beta$$

- Asymmetric Quant: clipping range $[\alpha, \beta]$ is not equally distributed around zero & has to be determined by calibration.

↳ min & max values of r can be used for calibration.
~~min~~ r could be param. of layer to Quantise

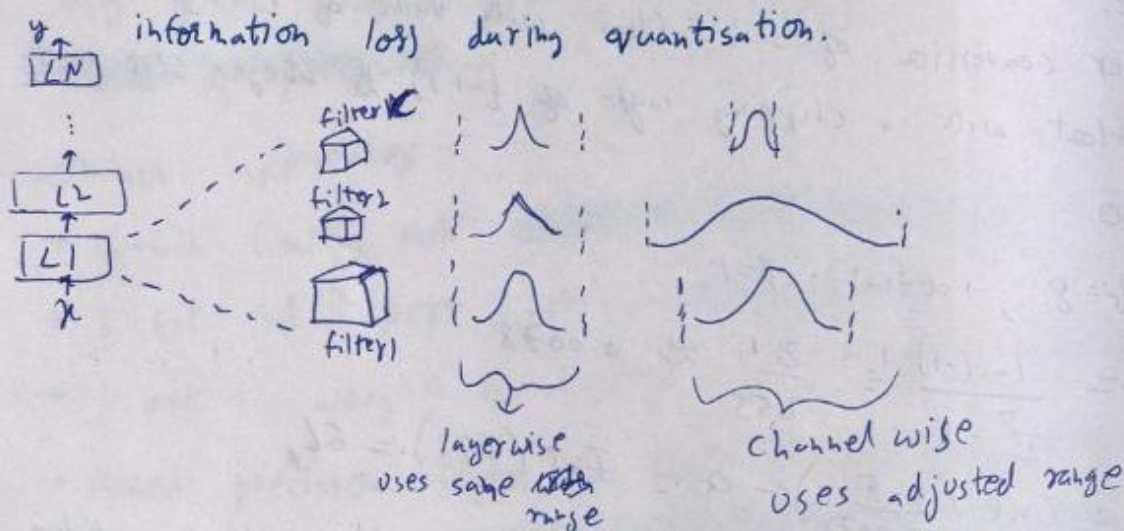
$$\alpha = r_{\min} \quad \& \quad \beta = r_{\max}$$

Layerwise & Channel wise Quantisation

- layerwise Quant: uses a single clipping range for all filters in a layer.

- channelwise quant: clipping range is adjusted separately for each channel.

↳ this is more efficient use of available bit width & less information loss during quantisation.



Example:

Q. consider a neural network with 2 filters/channels. Quantise the weights to 8-bit int. format. Assume symmetric quant. & weights to be

Filter 1: $[0.1, -0.05, 0.2, 0.08]$

Filter 2: $[1.5, -0.8, 2.1, -1.2]$

with $z=0$

Sol: layerwise quantisation
~~Find overall Min & Max from layer~~
 Max weight = 2.1 (from F2)
 Min " = -1.2 (from F2)
 hence $\alpha = \frac{-1.2}{2}$ & $\beta = \frac{2.1}{2}$

$$s = \frac{2.1 - (-1.2)}{2^8 - 1} = \frac{3.3}{255} \approx 0.01294$$

Then do Quantisation for every weight

$$q(r) = \text{Int}\left(\frac{r}{s}\right) \div 2$$

For filter 1

$$\text{Int}\left(\frac{0.1}{0.01294}\right) = 7$$

$$\text{Int}\left(\frac{-0.05}{0.01294}\right) = -3$$

$$\text{Int}\left(\frac{0.2}{0.01294}\right) = 15$$

$$\text{Int}\left(\frac{0.08}{0.01294}\right) = 6$$

hence quantised

filter 1: [7, -3, 15, 6]

For filter 2

$$\text{Int}\left(\frac{1.5}{0.01294}\right) = 115$$

$$\text{Int}\left(\frac{-0.8}{0.01294}\right) = -61$$

$$\text{Int}\left(\frac{2.1}{0.01294}\right) = 162$$

$$\text{Int}\left(\frac{-1.2}{0.01294}\right) = -92$$

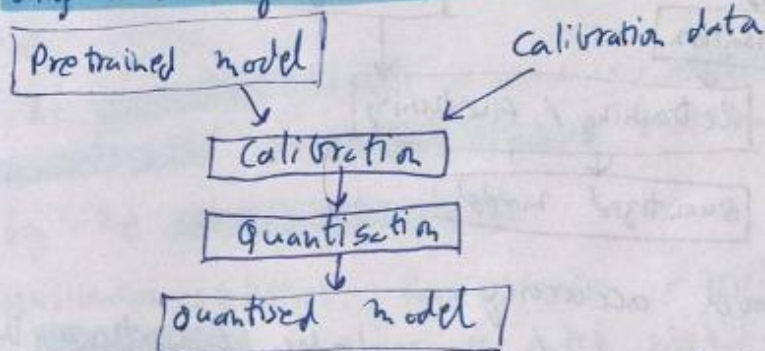
Quantised filter 2:

[115, -61, 162, -92]

Training based Quant. strategies

Post-Training Quantisation (PTQ)

- Model is quantised w/o fine tuning, reducing the needed data, which are only necessary for the calibration.



- doesn't apply fine-tuning to model param., reducing computational & time overhead.
- model forgets knowledge due to reduced bit width (lower bit width = lower accuracy)

- PTQ requires only a subset of training-set. It can even work with unlabelled data during the calibration.

In the calibration the clipping range $[\alpha, \beta]$, scaling factor S & zero offset Z are calculated.

- Models quantised with PTQ have often lower accuracy v/s quantisation aware training.

- PTQ has minimal computational overhead.

- Linear quantisation takes min/max as α/β , leading to high quantisation errors. i.e. clipping range is stretched to include outliers, since no. of levels is fixed it results in increased dist b/w levels.

- clipping defines α/β therefore ignores outliers.

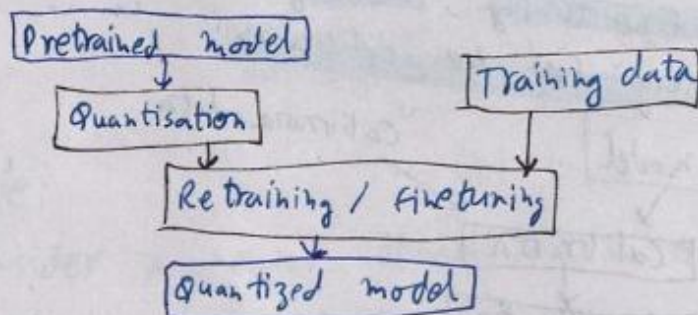
- Outlier channel splitting (OCS): doubles predefined number of channels to involve outliers in the quantisation.

Later the split channels are added & multiplied by 0.5. weights are split by the foll. eq:

$$OCS(w) = \begin{cases} (w - 0.5)/2 \\ (w + 0.5)/2 \end{cases}$$

Quantisation Aware Training (QAT)

After quantisation, the model is fine-tuned with the training data & model regains the loss of knowledge.



- PTQ drops model accuracy.

Sol: to recover lost knowledge finetune the quantised model.

- QAT needs access to the training data.

- standard training not possible for quantised models in integer precision because of the non-differentiable quantisation operator. (*)
- QAT uses float precision for fwd & backward pass to perform weight update. After weight update, the weights are pseudo-quantised, i.e. they are represented by float but have the value of a quantised weight. The fwd pass is performed by pseudo-quantised weights.
- Gradients approximated using straight through Estimator (STE). Fwd pass performed with the quantised model. For Backward pass, rounding operator approximated with identity func.

(*) sol: handle Fwd & rev. pass differently.

Perf. quantisation in fwd pass.

In backward pass, STE approximates rounding function with an identity function. This allows grad. from subsequent layers to pass through the quantisation step, enabling weight updates to continue.

Data-free Quantisation

- Training data isn't always available to determine the clipping range or to finetune the quantised model.
- real data is replaced by synthetic data which can be generated in different ways.
 - 1) GAN: the pretrained model is used as discriminator to train GAN to produce images that can be classified by the pre-trained model.
 - 2) Batch normalisation statistics: Backpropagation directly on a noise image using stored mean & var. of batch normalisation layers to calculate loss.

- adv. of batch normalisation statistics approach over GAN method the distr. of real training set is taken into account in the synthetic images.
- ZeroQ* is a data-free mixed precision quantisation approach.
- batch normalisation statistics are used to generate synthetic images by minimising the following condition

$$\min_{x^r} \sum_{i=0}^L \| \tilde{\mu}_i^r - \mu_i \|_2^2 + \| \tilde{\sigma}_i^r - \sigma_i \|_2^2$$

$x^r \rightarrow$ synthetic image
 $\tilde{\mu}_i^r / \tilde{\sigma}_i^r \rightarrow$ mean & std. deviation of synthetic image
 $\mu_i / \sigma_i \rightarrow$ stored in batch normalisation.
- precision of each layer is determined by a sensitivity measure where layers with high sensitivity have higher precision than layers with a low sensitivity.
- sensitivity is calculated with KL-div. w/o original model & model where the specific layer is quantized with the desired precision.

Parameter Pruning

- NN are over-parametrised & some parameters are unnecessary or redundant
- goal of pruning is to reduce number of model parameters by removing redundant / unnecessary parameters.
- pruning affects model performance & knowledge can be forgotten.
- pruning a specific layer also influences the following layers.
- pruning can also be grouped into diff. categories based on

What to prune?

When to prune?

How to prune?

How often to prune?

Structured Pruning

- removes whole channels from NN (also removes connected structures)
- reduces execution time on std. hardware because of fewer kernels.

Local pruning

- removes same amount of param. in each layer of NN.
ex: 20% channels in each layer

Magnitude-based Pruning

- assumes larger weights have higher influence on overall n/w output.
- hence bigger weights are pruned less than smaller weights.
- L1 pruning is mag. based, where for each channel, sum of weights is calculated & channels with smaller sums are removed during pruning.

Example: Magnitude based pruning

Q. consider pruning off 1 channel of full NN ~~layer~~ with 3 channels, each having weights

$$ch1: [0.1, -0.2, 0.15]$$

$$ch2: [0.8, 0.9, 0.7]$$

$$ch3: [0.05, -0.01, 0.03]$$

sol: calc. L1 norm 1st

$$L1 \text{ norm } ch1 = |0.1| + |-0.2| + |0.15| = 0.45$$

$$L1 \text{ norm } ch2 = |0.8| + |0.9| + |0.7| = 2.4$$

$$L1 \text{ norm } ch3 = |0.05| + |-0.01| + |0.03| = 0.09$$

→ hence remove ch3 with smallest norm

Unstructured Pruning

- removes single (sparse) weights
removes unconnected structures.
- difficult to optimise on std. hardware because of scattered nature of computations. Also req. special libraries for sparse data.

Global pruning

- deletes desired % of param. across the n/w.
pruning ratio may differ b/w layers.

ex: prune Overall 20% of the channels

Gradient-based Pruning

- uses gradients to decide which param. to prune.
- compute gradients using training dataset & the loss function.
- use mean gradients of a feature map to decide whether the corresponding channel should be pruned.
- Channels with low gradients are pruned.

Learning-based Pruning

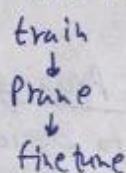
- importance of the parameters is learned.

ex: GMP, which uses a gating func. to decide if a channel should be pruned.

- during training gating func. is pushed to 0/1 for diff channels & channels with 0 as gate can be removed after training.

One-shot pruning

- desired pruning sparsity is achieved in 1 step & all param. to be pruned are removed in 1 step.



Pruning before training

- results in reduced computational complexity because pre-training can be skipped & only pruned W is trained

- SNIP prunes a randomly initialised W in an unstructured way by calc. sensitivity for each weight w with grad. g .

↳ it propagates one batch D through the randomly initialised W & calc. loss to get gradients g . sensitivity s_j for weight j is calc. as

$$s_j = \frac{|g_j(w; D)|}{\sum_{k=1}^m |g_k(w; D)|} ; m \rightarrow \text{total \# of parameters}$$

- if grad. has high absolute value it has high importance on overall result & should be kept.
- weights with smallest sensitivity are removed from model.

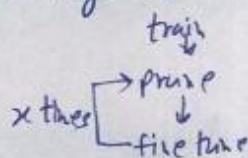
Information-based pruning

- HRank $\rightarrow$ example

- in HRank, for each o/p feature map the matrix rank is calculated.
- feature map is imp. if it has high rank & hence remove weights corresponding to feature maps with a low rank.

Iterative pruning

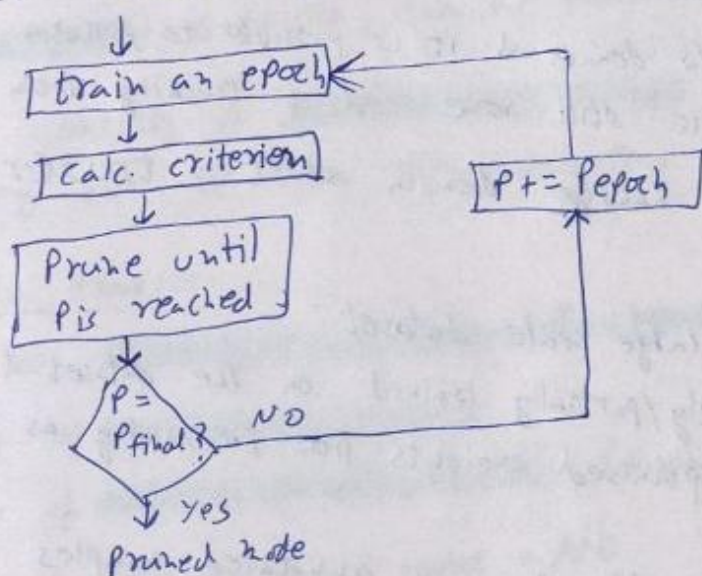
- Removes a small fraction of param. in each iteration until desired pruning sparsity is achieved.



Pruning during training

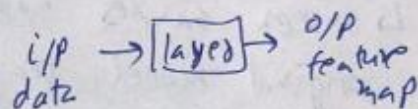
- n/w parameters are removed iteratively during training.
- pruning rate P is increased every epoch by P_{epoch} until final pruning rate P_{final} is reached.
- L1 norm is used as pruning criterion.

init. model, training dataset



Pruning after training

- param. pruned after pre-training & ^{then} pruned model is fine-tuned.
- Ex: HRank
 - ↳ not only considers internal knowledge but also ^{ip} data during training.
 - ↳ defines importance of channel according to matrix rank of o/p feature map.
 - ↳ rank calc with Singular Value Decomposition (SVD)
 - ↳ mini-batch rank calculated for each channel is enough due to low variance
 - ↳ remove channels with low rank.



Data-free Parameter Pruning

- Ex: DFNP do not need access to training data for fine-tuning.
- DFNP generates synthetic data with help of a generator & uses generated data in a student-teacher method to transfer knowledge from unpruned to re pruned model.
- generator is trained by using re unpruned model as a discriminator based on adversarial training.

when is data-free approach useful?

09- Few-Shot Learning

- if we have annotated samples, supervised learning is promising.
 - ↳ sample annotation is a **costly process**
 - ↳ sample available for a specific applications may be **limited**
- pretrained model **might not work on target test set** since it is trained on a source domain dataset that differs to some extent.
- when moving to new domain & it is possible to perform large scale annotation, there is still some cost of training from scratch.
- one approach to handle domain shift = transfer learning

Transfer learning

- pretrain with large scale dataset
- then NN is entirely/partially trained on the dataset & task of interest using optimised weights from pre-training as prior knowledge.

limitations:

- requires considerable no. of annotated samples

What if we don't have enough annotated samples?

Transfer learning is the approach of adapting a pre-trained model to the new task using the new dataset.

Few-Shot Learning (FSL)

- FSL addresses the **ability** of an already trained NN **to generalise to unseen samples**, using **only a few annotated samples**.
 - ↳ unseen samples from target domain **not encountered** during pre-training.
 - ↳ unseen samples may have **semantic overlap** with data used for original model training, but differ in context. ex: real dog images vs animated dog images.
 - ↳ unseen samples may belong to diff. obj. categories ex: wolfs instead of dogs.
- original model training = pretraining
- Like transfer learning it is also assumed to have access to large-scale annotated data during pre-training, referred to as **source** (D_S) & other unseen dataset from the task of interest referred to as **Target** (D_T).

terminology

- main difference is that D_T contains a **small dataset** with **only few-annotated samples** for adapting pre-trained model.
Also refer to it as **support set (S)**
set for testing = query set (Q)

- **Support set**: consists of few annotated samples of existing or new **class categorie(s)**, used to train the pre-trained model.
- **Query set**: consists of **samples from target domain** to be **tested**. They can be from new/existing class categorie(s).

FSL classification example:

- Also known as **N-way, K-shot few-shot classification task**

$N \rightarrow$ n.o. of **novel class categories**

$K \rightarrow$ n.o. of **annotated samples per class** (normally $K \leq 10$)

total samples to train pre-trained model = NK

$\hookrightarrow$ these samples form the **support set (S)**

- we have $S = \{s_n\}_{n=1}^N$

where $s_n = \{(x, y^n)_i\}_{i=1}^K$; $y^n \in \{1, \dots, N\}$

and query set $Q = \{(x_j)\}_{j=1}^L$

Note: S & Q share the same label space.

- **Categories** in D_T & D may **overlap**, hence we can encounter **new unseen samples of same class category** as in source set.

- **Categories** in D_T & D may also be **mutually exclusive**, hence we can encounter **new unseen samples of novel categories** than in source dataset.

Fewshot segmentation

- deal with **N-way K-shot problem** on the **pixel level**.

- D_T consists of support set $S = \{(x, y)_i\}_{i=1}^K$; $y \in \mathbb{R}^{H \times W}$ is the **pixelwise seg. map** & query set $Q = \{(x_j)\}_{j=1}^L$.

Types of approaches

- std. supervised learning has large n.o. of samples in the mini-batch to perform backpropagation & grad. descent, not possible in FSL due to lack of plethora of training data.
- supervised learning can easily lead to overfitting.
- 4 types of approaches in FSL
 - 1) Data-based approaches: try to increase limited dataset, ex using augmentation
 - 2) Parameter based approaches: constrain the way parameters change. Ex: meta-learning
 - 3) Metric based approaches: aim to learn a dist. func. b/w available samples. Ex: similarity function
 - 4) Gradient-based meta-learning approaches: based on a base learner & a meta-learner that cooperate to learn from tasks.

Meta learning

- refers to learning the learning algorithm itself. known as learning to learn.
- the main types of meta learning algo. are
 - metric-based meta learning
 - Model-based " "
 - optimisation-based " "
- common approach for meta learning algorithms is using tasks.
↳ we apply task-based (episodic) learning where model is optimised using tasks τ sampled from $\mathcal{D}$. This is diff. from mini-batch training of std. supervised learning.

Task formation

- consider N -way, k -shot classification with support & query sets.
- randomly select N classes from entire set of classes C .
Ex: if $C = [\text{dog, cat, bird, lion, ...}]$ & assume we select 2 classes at random ^{i.e. $N=2$} then we have classes $[\text{dog, lion}]$
- then select all samples from D that are classified as dog/lion, let this subset be $D^{\text{dog, lion}} = \{D^{\text{dog}}, D^{\text{lion}}\}$
- next select at random K -shots (samples) per class from $D^{\text{dog, lion}}$
- From $D^{\text{dog, lion}}$ we get support set $S^{\text{dog, lion}}$
- Finally, we remove all samples from $D^{\text{dog, lion}}$ to get query set i.e. $Q^{\text{dog, lion}} = D^{\text{dog, lion}} \setminus S^{\text{dog, lion}}$

Optimisation based Meta learning

- we have 2 models the meta-learner & base learner which will be updated differently.
- both models have same NN architecture, but
base learner is optimised using single task
meta learner " optimised using set of tasks, i.e.
it distills the info learned from several base learners to update its parameters.
- objective: find optimal parameters θ that allow learning using a few-shots only. [each task is composed of few shots only]
Ultimately this process allows efficient fine-tuning to the target task.
- based on this the foll. optim. based algo. for meta-learning have been developed
 - 1) Model Agnostic Meta Learning (MAML)
 - 2) Reptile

MAML

- is a **model based**, **task-agnostic** approach to learn adapting to a task using only a few gradient steps.

Pseudo Algo.

1) randomly init. θ

2) init. base model with meta-learner parameters θ . reptile

3) Outer loop: sample a batch of tasks, each task $\tau_i = \{s_i, \theta_i\}$

4) inner loop: Train **base model** to using s_i with grad. descent or any other optimiser for a few iterations to get updated param. θ_i' .

$$\theta_i' \leftarrow \theta - \alpha \nabla_{\theta} L_{\tau_i}(\theta)$$

$\alpha \rightarrow$ base model learning rate

$L_{\tau_i} \rightarrow$ **task loss** i.e. **cross entropy** for classification

5) back in outer loop: update meta-learner param

$$\theta \leftarrow \theta - \beta \nabla_{\theta} \sum_{\tau_i} L_{\tau_i}(\theta_i')$$

$$\theta \leftarrow \theta + \beta \frac{1}{n} \sum_{i=1}^n (\theta_i' - \theta)$$

$\lambda \rightarrow 2.0 \times \text{of sample tasks}$

$\beta \leftarrow$ meta learners learning rate

$L_{\tau_i}(\theta_i') \rightarrow$ calculated using **query sets** of sampled tasks θ_i' .

$L_{\tau_i}(\theta_i') \rightarrow$ function in the optimised base learner param. θ_i' using s_i .

Like calculating loss WRT validation set

- updating meta learner = **aggregating** info learned by individual base models on diff. tasks.

- ~~calc.~~ calc. grad. $\nabla_{\theta} \sum_{\tau_i} L_{\tau_i}(\theta_i')$ is **2nd order derivative** which is **expensive**.

- inner loop param. update
base learner is trained for k iter. $k \geq 1$ on task τ_i
i.e. $i=1$, $\theta_{1,0} = \theta$

$$\theta_{1,1} = \theta_{1,0} - \alpha \nabla_{\theta} L_{\tau_1}(f_{\theta_{1,0}})$$

doesn't need
query set
to update
param.

$$\theta_{1,2} = \theta_{1,1} - \alpha \nabla_{\theta} L_{\tau_1}(f_{\theta_{1,1}})$$

$$\dots$$

$$\theta_{1,k} = \theta_{1,k-1} - \alpha \nabla_{\theta} L_{\tau_1}(f_{\theta_{1,k-1}})$$

- afterwards, update ^{meta}param θ in outer loop

$$\theta \leftarrow \theta - \beta g_{MAML}$$

$$\text{where } g_{MAML} = \nabla_{\theta} L_{\tau_1}(f_{\theta_{1,k}})$$

$$\nabla_{\theta} L_{\tau_1}(f_{\theta_{1,k}}) = \nabla_{\theta_{1,k}} L_{\tau_1}(f_{\theta_{1,k}}) \cdot (\nabla_{\theta_{k-1}} \theta_k) \dots (\nabla_{\theta_0} \theta_1) \cdot (\nabla_{\theta} \theta_0)$$

Factorising we get,

$$g_{MAML} = \nabla_{\theta_{1,k}} L_{\tau_1}(f_{\theta_{1,k}}) \prod_{l=1}^k (I - \alpha \nabla_{\theta_{l-1}} (\nabla_{\theta} L(f_{\theta_{1,l-1}})))$$

^{2nd order derivatives}

- to relax computations, reformulate MAML as 1st order derivative approach by ignoring the chain rule. First order MAML (FOMAML)

$$\text{hence } g_{FOMAML} = \nabla_{\theta_{1,k}} L_{\tau_1}(f_{\theta_{1,k}})$$

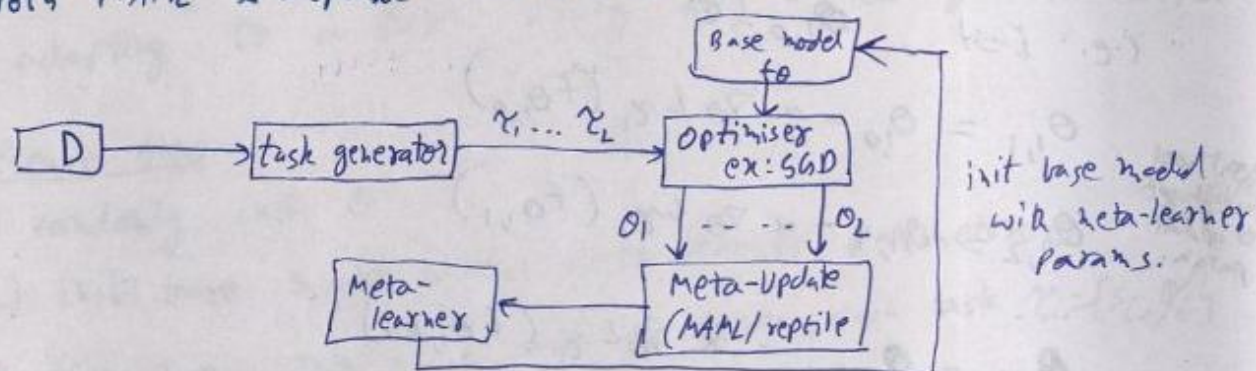
- results indicate no significant performance drop with 1st order derivatives.

Reptile (1st order approach)

- another optimization-based meta learning algo. like MAML.
- only differs in meta-update rule as it doesn't require 2nd order derivatives & hence is more computationally efficient.
- depends on learning from tasks, idea of base- & meta learners is like MAML.
- the meta parameters update rule can be thought as averaging gradient of different tasks.
↳ related to SGD but not like averaging gradients.

optimisation based approaches

- both MAML & Reptile share same idea



- both MAML & Reptile aim to find parameters that allow meta-learner to efficiently adapt to a new task from target samples.
- both are task & model agnostic.
- importantly Reptile doesn't req. query set to update meta-param.
- both algo. are sensitive to hyperparameters & need to be tuned carefully.

EX: FS-MICS [refer slides]

(Few-shot microscopy image cell segmentation)

mIoU (Mean Intersection over Union)

$$mIoU = \frac{y' \cap y}{y \cup y}$$

$y' \rightarrow$ prediction

$y \rightarrow$ ground truth segmentation

10 - Uncertainty estimation

- ip to NN can be manipulated s.t. the model predicts it as a known class with high confidence, hence being easily fooled.

Uncertainty

- op confidence alone isn't enough to conclude on certainty of the model.
- In critical cases it is imp. to quantify NN's predictive uncertainty. Ex: automated driving, robot based medical operations.

Uncertainty estimation

- given training dataset $D_{\text{train}} \{x, y\}_{i=1}^N$

- NN takes as ip x (ex: image) $x \in \mathbb{R}^D$ with D -dimensional features.

- in classification y is one of K -classes $y \in \{1, \dots, K\}$ represented by a 1-hot vector

- in regression y is real valued with arbitrary dimension

ex: $y \in \mathbb{R}^D$ in depth estimation

- given training set model is optimised with predefined cost function

$$\min_{\theta} E_{x,y} L(f(x; \theta), y) \approx \frac{1}{N} \sum_{i=1}^N L(f(x_i; \theta), y_i)$$

- in both classification & reg. we seek prediction likelihood $p(y|x, \theta)$

in classification $p(y|x, \theta)$ is defined as $\text{softmax} \left(\frac{f(x; \theta)}{\exp(\sigma^2)} \right)$

in reg. $p(y|x, \theta)$ is assumed to be gaussian distr. $\mathcal{N}(f(x; \theta), \Sigma)$

↳ both problems face problem of uncertainty estimation.

Types of uncertainty

→ aleatoric uncertainty:
(AU)

ambiguity arising from training dataset D due to intrinsic randomness represented by ϵ / Σ .

(EU)
epistemic uncertainty:

ambiguity resulting from the model parameters θ .

↳ science in Greek

Aleatoric Uncertainty

- also known as **data uncertainty**
- Uncertainty inherent in data distr. from **noise, occlusions or reflections or sensor noise**
- **characteristic of data distr.**
- is **irreducible**
- **AU** is function of model input
- further distinguished as

1) Homoscedastic uncertainty

- **constant regardless of inputs** i.e. doesn't change for diff. i/p.
- it is **image-independent** but **task-dependent AU**.

2) Heteroscedastic uncertainty

- **changes for diff. i/p** ex: noise level is higher for certain i/p than for other i/p.
- it is **image-conditional AU**.

Epistemic Uncertainty

- known as **model / knowledge uncertainty**
- stems from **missing knowledge** & thus occurs for **too few training data** or if data is **uninformative**.
- **reducible** with more diverse training data.
- EU is function of the model i/p.

Predictive Uncertainty (PE)

- **combination of ~~AU~~ AU & EU**

$$PE = AU + EU$$

PE ~~estimation~~ estimation:

- AU is function over model i/p
- AU can be **modelled as func. over model o/p.**

- it can thus be accounted for using PE estimation methods.
- Predictive methods assume a parametric distr. over the model o/p.
- model can then be optimised by maximising the log-likelihood of the target distr. as:

$$\log p(D|\theta) = -\frac{1}{N} \sum_{i=1}^N \log p(y_i | x_i, f(x_i; \theta))$$

PE for classification

- scoring rule measures the quality of predictive uncertainty
- in classification, log-likelihood is given by the softmax cross-entropy loss, which is a suitable scoring rule.
- prediction likelihood is obtained by sampling from the resulting probability vector

$$p(y|x, \theta) = \text{softmax}(f(x; \theta))$$

- uncertainty is then estimated using entropy of the resulting prob. vector with C classes as:

$$H(p) = - \sum_{c=1}^C p_c \log p_c$$

PE for regression

- we assume training set can be modeled as

$$d(x) = \underset{\substack{\downarrow \\ \text{model op}}}{f(x)} + \underset{\substack{\downarrow \\ \text{observation noise in data}}}{h(x)}$$

- obs. noise can be modelled interpreted & modeled as the std. deviation σ .
- homoscedastic uncertainty methods assume const. obs. noise σ for all variables, i.e. const. noise regardless of i/p
- heteroscedastic uncertainty can be captured by tuning the noise parameter σ , i.e. noise based on the i/p.
↳ noise param. can be learned as a func. of the data
- h/w o/p can be interpreted as mean $\mu(x)$ of the noisy target distr.
- model predicts the output, mean $\mu(x)$ & var. $\sigma^2(x)$ of target distr.

- For this reason, model is optimised to o/p the param. of a distr. by maximising the log-likelihood of the target distr.

- When model optimises the L_2 loss, o/p distr. can be modelled as Gaussian Distr.

- prediction likelihood $p(y|x, \theta) = \mathcal{N}(f(x; \theta), \Sigma(x))$
 $\Sigma(x) \Rightarrow \text{diag}(\sigma(x)^2) \rightarrow$ is label noise

$\rightarrow$ assumption made is that
 $n(x) \sim \mathcal{N}(0, \Sigma(x))$

i.e. data noise follows a Gaussian distr. with 0 mean.

- uncertainty aware loss is then

$$L_{uct} = \frac{1}{N} \sum_{i=1}^N \frac{(y_i - \mu(x_i))^2}{2 \sigma(x_i)^2} + \frac{1}{2} \ln \sigma(x_i)^2$$

$\ln(\cdot)$ term in loss can lead to instability

- hence var. $\sigma(x)^2$ is substituted with logarithmic variance

$$\sigma_{\log}(x)^2 = \log(\sigma(x)^2)$$

which makes uncertainty aware loss

$$L_{uct} = \frac{1}{N} \sum_{i=1}^N \frac{(y_i - \mu(x_i))^2}{2 \exp(\sigma_{\log}(x_i)^2)} + \frac{1}{2} \sigma_{\log}(x_i)^2$$

$\hookrightarrow$ hence no division by 0.

finally aleatoric uncertainty

$$U_{aleatoric} = \sigma(x)^2$$

- When model optimises the L_1 loss, o/p distr. can be modeled as Laplace distr.

- uncertainty aware loss is then

$$L_{uct} = \frac{1}{N} \sum_{i=1}^N \frac{|y_i - \mu(x_i)|}{\sigma(x_i)} + \ln \sigma(x_i)$$

- Finally AU is $U_{aleatoric} = 2 \sigma(x)^2$

- like for Gaussian uncertainty loss, $\sigma(u)$ can be replaced by $\sigma_{\log}(u) = \ln(\sigma(u))$ for numerical stability.

Empirical uncertainty estimation

- Epistemic uncertainty is a func. of the model, hence it can be modelled as a func. over model parameters.
prior over model weights is assumed: $\sigma \sim \mathcal{N}(0, 1)$
- This is also called a Bayesian NN.
 ↳ deterministic weights are replaced by distr. over the parameters.
 ↳ then it is averaged over all possible w/w weights, referred to as marginalisation
 ↳ Bayesian inference used to compute posterior over the model weights $p(\theta|x, y)$
- since Bayesian NN are impractical for inference, several approximate solutions such as MC dropout exist.
- Alternate sol. are ensemble methods, which are empirical approaches to model distr. of parameters.
- approximation of posterior param. distr. $p(w|x, y)$ can be done during inference, assuming for ex: a gaussian prior distr. for model weights $w \sim \mathcal{N}(0, \sigma)$

Ensemble methods

- predictions obtained with M models, with corresponding parameters $\{\theta_m\}_{m=1}^M$.
- Classification: prob. o/p is obtained by averaging over the M models:

$$p = \frac{1}{M} \sum_{m=1}^M p_m \quad ; p_m \rightarrow \text{prob. vector of } m^{\text{th}} \text{ model}$$

epistemic uncertainty is then the entropy of resulting prob. vector:

$$H(p) = - \sum_{c=1}^C p_c \log p_c$$

- Regression: empirical μ & σ^2 are computed from M predictions as

$$\text{Mean } \mu = \frac{1}{M} \sum_{i=1}^M \hat{y}_i$$

$$\text{Var } \sigma^2 = \frac{1}{M} \sum_{i=1}^M (\hat{y}_i - \mu)^2$$

Bootstrap Ensembles

- train M models with same architecture
- train models on subsets of entire training data by bootstrapping samples.
- OR train models on entire training set but with diff. random weight initialisation & random data shuffling.
- NN perform better with more training data.

Dropout Ensembles

- dropout used as regularisation during training & for inference to predict compute prediction uncertainty.

dropout randomly drops units with prob. p_{drop} .

- dropout can be interpreted as ensemble with shared weights. Diff. models are sampled by randomly dropping model units.
- During inference M model configurations are used to calc. mean & variance by using M diff. dropout config.

Snapshot Ensembles

- snapshot ensembles remove the necessity of training M models.
- Possible difficulties using single training run: resulting models could be too similar.
- M models are trained using cyclic LR scheduling.

Radical change of LR during the training run.

- model undergoes several LR annealing cycles. it converges & escapes several local minima.

Combining Predictive & Empirical methods

- both can be combined to estimate aleatoric & epistemic uncertainty in a single model.
- model is optimised using uncertainty-aware loss L_{uct} to predict the mean μ & var. σ^2 as a measure of aleatoric uncertainty.
- in addition, distr. over model weights is approximated using dropout ensembles.

- uncertainty aware loss

$$L_{uct} = \frac{1}{E} \sum_{j=1}^E \frac{(y_j - \mu(x_j))^2}{2\sigma(x_j)^2} + \frac{1}{2} \ln \sigma(x_j)^2$$

$E=1$ for single-valued regression task

= h.o. of o/p pixels; for dense regression tasks

- in practice, var. $\sigma(x)^2$ is substituted by logarithmic variance

$$\sigma_{\log}(x)^2 = \log(\sigma(x)^2)$$

hence, $L_{uct} = \frac{1}{E} \sum_{j=1}^E \frac{(y_j - \mu(x_j))^2}{2 \exp(\sigma_{\log}(x_j)^2)} + \frac{1}{2} \sigma_{\log}(x_j)^2$

regularises loss to not predict infinite uncertainty

- L_{uct} is built with 2 components

1) residual regression, obtained by stochastic sampling through R model
→ uncertainty over parameters

2) uncertainty regularisation term → variance is learned with the loss func.

- mean & var. for a single i/p x & M models sampling steps is

$$\mu = \frac{1}{M} \sum_{i=1}^M \mu_i(x)$$

$$\sigma^2 = \frac{1}{M} \sum_{i=1}^M (\mu_i(x) - \mu)^2 + \sigma_f^2(x)$$

- term $\exp(\sigma_{\log}(x)^2)$ in L_{uct} makes model more robust to noisy data (heteroscedastic uncertainty)

- all classification models have homoscedastic uncertainty (data indep. uncertainty)
- hence, classification model is modified to marginalize over intermediate heteroscedastic regression uncertainty placed over logit space.
- classification model $f(\cdot; \theta)$ predicts a vector of unaries f_i for each pixel.
- f_i is then passed through softmax to get prob. vector p_i .

$$p_i = \text{softmax}(f_i)$$

- model is changed by placing a Gaussian distr. over unaries vector

$$\hat{x}_i | w \sim \mathcal{N}(f_i^w, (\sigma_i^w)^2)$$

$$\hat{p}_i = \text{softmax}(\hat{x}_i)$$

f_i^w & $(\sigma_i^w)^2$ are n/w o/p with param. w .

f_i^w is corrupted with gaussian noise $(\sigma_i^w)^2$. This corrupted vector is passed through softmax to obtain prob. vector $\hat{p}_i$ for pixel i .

- expected log-likelihood for observed class c is then

$$\log E_{\mathcal{N}(\hat{x}_i; f_i^w, (\sigma_i^w)^2)} [\hat{p}_{i,c}]$$

↳ loss func. for model training

- since no analytical sol. exists for integration over gaussian distr. obj. is approximated using Monte carlo integration by sampling unaries over softmax function.

↳ fast in practice. only required to sample from logits

- numerically stable loss with c' being the c^{th} element in logit vector $\hat{x}_{i,n}$ is

$$\hat{x}_{i,n} = f_i^w + \sigma_i^w \epsilon_n ; \epsilon_n \sim \mathcal{N}(0, I)$$

$$L_X = \sum_i \log \frac{1}{\sum_m \exp(\hat{x}_{i,m,c} - \log \sum_{c'} \exp \tilde{x}_{i,m,c'})}$$